

2025 - 2026

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : SYSTÈME D'INFORMATION MOBILE

**Application Mobile intelligente de
gestion du stationnement en Tunisie**

Réalisé par: Ghayeth Arbi

Encadrant ESPRIT: Rim Ghribi

Encadrant Entreprise: Mehdi Kaouech



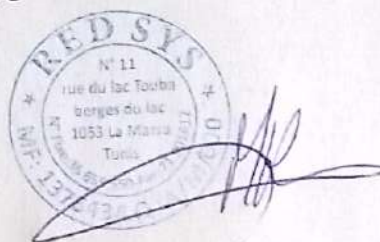
Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous:

- Nom & Prénom : Ghayeth Arbi

Encadrant Entreprise

- Nom & Prénom : Mehdi Laouech

Cachet & Signature



Encadrant Académique

- Nom & Prénom : Rim Ghribi

Signature

Ce formulaire doit être rempli, signé et scanné.

Ce formulaire doit être introduit après la page de garde.

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à ma mère pour son amour, sa patience, ses sacrifices et son soutien indéfectible tout au long de mon parcours. Ses encouragements, sa confiance et les valeurs qu'elle m'a transmises ont été une source de motivation permanente et ont largement contribué à l'aboutissement de ce projet.

J'adresse également mes sincères remerciements à ma tante, ainsi qu'à mes frères et sœurs, pour leur soutien, leur présence et leurs encouragements tout au long de mes études. Leur confiance et leur affection m'ont donné la force de poursuivre mes objectifs et de surmonter les difficultés rencontrées.

Je remercie également mes amis pour leur soutien moral, leurs conseils et les nombreux moments de partage qui ont rendu cette expérience plus agréable. Leur présence a été une véritable source de motivation tout au long de ce projet.

Je souhaite exprimer ma sincère reconnaissance à mon entreprise d'accueil, **RedSys**, pour m'avoir offert l'opportunité de réaliser ce Projet de Fin d'Études dans un environnement professionnel stimulant. Ce stage m'a permis de mettre en pratique les connaissances acquises au cours de ma formation, de renforcer mes compétences techniques et de participer au développement de l'application intelligente de gestion du stationnement **TuniPark**.

Je souhaite exprimer ma profonde gratitude envers mon superviseur chez RedSys, **M. Mehdi Kaouech**, pour son soutien, sa présence constante, sa confiance et ses conseils précieux tout au long de ce projet. L'expertise, les commentaires constructifs et le soutien toujours présent de ce dernier ont considérablement participé à l'accomplissement de ce travail.

Je remercie chaleureusement toute l'équipe de **RedSys** pour son accueil, sa disponibilité, son professionnalisme et l'excellente ambiance de travail qui ont rendu cette expérience particulièrement enrichissante.

Je tiens aussi à exprimer mes plus profonds remerciements à mon encadrante universitaire, **Mme Rim Ghribi**, pour son encadrement, sa présence constante, ses recommandations et son soutien tout au long de la pfe.

Enfin, je remercie toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet et m'ont soutenu durant cette étape importante de mon parcours.

Résumé

Ce mémoire présente les travaux réalisés dans le cadre d'un projet de fin d'études en vue de l'obtention du diplôme d'ingénieur en informatique. Le projet, intitulé « **TuniPark : Plateforme intelligente de gestion et de recommandation de parkings** », a pour objectif de répondre aux difficultés rencontrées par les conducteurs lors de la recherche de places de stationnement ainsi qu'au manque de solutions numériques permettant aux propriétaires de parkings privés de gérer efficacement leurs espaces. Afin de répondre à ces problématiques, une plateforme complète composée d'une application mobile et d'une interface web d'administration a été conçue et développée. La solution permet aux utilisateurs de rechercher des parkings publics et privés, de gérer leurs sessions de stationnement, de procéder au paiement en ligne et aux propriétaires de publier et administrer leurs parkings. Elle intègre également un module d'IA basé sur un modèle **Random Forest**, capable de recommander les parkings les plus adaptés en fonction de plusieurs critères tels que la distance, la disponibilité, le comportement des utilisateurs et les tarifs proposés. Cette plateforme contribue ainsi à améliorer l'expérience utilisateur, à optimiser l'utilisation des places de stationnement et à faciliter l'administration de l'ensemble du système.

Mots-clés : TuniPark, Gestion de parkings, Application mobile, Flutter, Angular, NestJS, PostgreSQL, Prisma, IA, Random Forest, FastAPI, Flouci, Firebase Cloud Messaging.

Table des matières

Table des figures	10
Liste des tableaux	12
Introduction Générale	13
1 Contexte Général et Étude Préalable	15
1.1 Introduction	15
1.2 Entreprise d'accueil	15
1.2.1 Aperçu de RedSys	15
1.2.2 Domaines d'activité de RedSys	16
1.2.3 Contexte organisationnel	16
1.3 Cadre général du projet	17
1.4 Problématique	17
1.5 Étude de l'existant	17
1.6 Critique de l'existant	18
1.7 Solution proposée	18
1.8 Méthodologie de développement	19
1.8.1 Méthodologie Agile	19
1.8.2 Framework SCRUM	20
1.9 Conclusion	20
2 Analyse et Spécification des Besoins	21
2.1 Introduction	21
2.2 Identification des acteurs	21
2.2.1 Utilisateur	21
2.2.2 Administrateur	22
2.3 Besoins fonctionnels	22
2.4 Besoins non fonctionnels	23
2.5 Diagramme global des cas d'utilisation	23
2.6 Description des principaux cas d'utilisation	25

2.6.1	Authentification	25
2.6.2	Gestion d'une session de parking	26
2.6.3	Gestion d'un parking privé	27
2.6.4	Paiement d'une session	28
2.6.5	Validation des parkings privés	29
2.7	Product Backlog	29
2.8	Conclusion	31
3	Conception et Architecture du Système	32
3.1	Introduction	32
3.2	Architecture du système	32
3.2.1	Architecture logique	32
3.2.2	Architecture physique	34
3.3	Choix techniques	36
3.3.1	Flutter	36
3.3.2	Angular	36
3.3.3	NestJS	37
3.3.4	PostgreSQL et Prisma	37
3.3.5	FastAPI et Random Forest	37
3.3.6	Authentification JWT	38
3.3.7	Cloudinary	38
3.3.8	Firebase Cloud Messaging	39
3.3.9	Flouci	39
3.3.10	Nodemailer	39
3.4	Conception de la base de données	39
3.4.1	Tables de gestion des utilisateurs et des accès	40
3.4.2	Tables de gestion des parkings	41
3.4.3	Tables de gestion des sessions de stationnement	41
3.4.4	Table de gestion des paiements	41
3.4.5	Tables de gestion des notifications	42
3.4.6	Table utilisée par le système de recommandation	42
3.4.7	Diagramme de classes	42
3.5	Diagrammes de séquence	44
3.5.1	Diagramme de séquence de l'authentification	44
3.5.2	Diagramme de séquence du renouvellement automatique du jeton	46
3.5.3	Diagramme de séquence de la création d'une session de sta- tionnement	47
3.5.4	Diagramme de séquence du paiement d'une session de sta- tionnement	47

3.5.5	Diagramme de séquence de la recommandation intelligente des parkings	48
3.5.6	Diagramme de séquence de l'ajout d'un parking privé	51
3.5.7	Diagramme de séquence des notifications	53
3.6	Conclusion	55
4	Réalisation et Implémentation	56
4.1	Introduction	56
4.2	Environnement de développement	56
4.3	Implémentation du Backend	58
4.3.1	Module d'authentification	58
4.3.2	Module de gestion des parkings	59
4.3.3	Module de gestion des tarifs	60
4.3.4	Module de gestion des sessions de stationnement	61
4.3.5	Module de paiement	62
4.3.6	Module de recommandation intelligente	63
4.3.7	Module de notifications	64
4.3.8	Module des interactions avec les parkings	65
4.3.9	Module de gestion des utilisateurs	66
4.4	Implémentation de l'application mobile	67
4.4.1	Authentification	67
4.4.2	Exploration des parkings	68
4.4.3	Création d'une session de stationnement	69
4.4.4	Paiement d'une session	70
4.4.5	Suivi de la session	71
4.4.6	Ajout d'un parking privé	72
4.4.7	Profil utilisateur	73
4.4.8	Notifications	74
4.5	Fonctionnalités administratives	75
4.6	Présentation du déploiement	76
4.7	Conclusion	78
5	Tests, Validation et Résultats	79
5.1	Introduction	79
5.2	Stratégie de test	79
5.3	Outil de test des API	80
5.4	Validation des API du Backend	81
5.5	Tests fonctionnels	81
5.5.1	Tests d'authentification et de contrôle d'accès	82
5.5.2	Tests de gestion des parkings	82
5.5.3	Tests des sessions de stationnement et des paiements	83

5.5.4	Tests des notifications	83
5.5.5	Tests du système de recommandation	83
5.6	Validation de la base de données	84
5.7	Validation des services externes	85
5.7.1	Validation des paiements avec Flouci	85
5.7.2	Validation des notifications Firebase	86
5.7.3	Validation du stockage Cloudinary	87
5.8	Résultats et discussion	88
5.9	Conclusion	89
Conclusion Générale et Perspectives		90
Bibliographie		92

Table des figures

1.1	Logo de RedSys	16
1.2	Icône de l'application TuniPark	18
1.3	Cycle de développement Agile	19
2.1	Diagramme global des cas d'utilisation de l'application TuniPark . .	24
3.1	Architecture logique de TuniPark	34
3.2	Architecture physique de l'application TuniPark	35
3.3	Logo du framework Flutter	36
3.4	Logo du framework Angular	36
3.5	Logo du framework NestJS	37
3.6	Logo de PostgreSQL	37
3.7	Microservice IA développé avec FastAPI	38
3.8	Authentification basée sur JWT	38
3.9	Logo de Cloudinary	38
3.10	Logo de Firebase Cloud Messaging	39
3.11	Logo de Flouci	39
3.12	Logo de Nodemailer	39
3.13	Diagramme de classes de l'application TuniPark	43
3.14	Authentification	45
3.15	Renouvellement automatique du jeton	46
3.16	Création d'une session de stationnement	47
3.17	Paiement d'une session de stationnement	48
3.18	Recommandation intelligente des parkings	50
3.19	Ajout d'un parking privé	52
3.20	Envoi des notifications	54
4.1	Implémentation du module d'authentification	59
4.2	Implémentation du module de gestion des parkings	60
4.3	Implémentation du module de gestion des tarifs	61
4.4	Implémentation du module des sessions de stationnement	62

4.5	Implémentation du module de paiement	63
4.6	Implémentation du module de recommandation intelligente	64
4.7	Implémentation du module de notifications	65
4.8	Implémentation du module des interactions avec les parkings	66
4.9	Implémentation du module de gestion des utilisateurs	67
4.10	Interfaces d'authentification	68
4.11	Exploration des parkings	69
4.12	Création d'une session de stationnement	70
4.13	Paiement d'une session	71
4.14	Suivi d'une session active	72
4.15	Ajout d'un parking privé	73
4.16	Interface du profil utilisateur	74
4.17	Interface des notifications	75
4.18	Tableau de bord de l'administration	76
4.19	Interface de gestion des ressources	76
4.20	Déploiement du Backend et de la base de données sur Render	77
4.21	Déploiement de la plateforme web d'administration sur Vercel	77
5.1	Documentation et tests des API à l'aide de Swagger	80
5.2	Validation de l'API d'authentification avec Swagger	81
5.3	Validation des données enregistrées dans PostgreSQL	84
5.4	Validation d'un paiement avec Flouci	86
5.5	Validation des notifications Push avec Firebase Cloud Messaging	87
5.6	Validation du stockage des images avec Cloudinary	88

Liste des tableaux

2.1	Authentification	25
2.2	Gestion d'une session de parking	26
2.3	Gestion d'un parking privé	27
2.4	Paie ment d'une session	28
2.5	Validation des parkings privés	29
2.6	Product Backlog de l'application TuniPark	29
3.1	Tables de gestion des utilisateurs et des accès	40
3.2	Tables de gestion des parkings	41
3.3	Table de gestion des sessions de stationnement	41
3.4	Table de gestion des paiements	41
3.5	Tables liées aux notifications	42
3.6	Table utilisée par le système de recommandation	42
4.1	Principaux outils et technologies utilisés lors de l'implémentation de TuniPark	57
5.1	Tests d'authentification et de contrôle d'accès	82
5.2	Tests de gestion des parkings	82
5.3	Tests des sessions de stationnement et des paiements	83
5.4	Tests des notifications	83
5.5	Tests du système de recommandation	84

Introduction Générale

Avec l'augmentation du nombre de véhicules et l'urbanisation croissante, la recherche d'une place de stationnement est devenue un véritable défi dans les grandes villes. Les conducteurs perdent souvent un temps considérable à chercher un parking disponible, tandis que de nombreux propriétaires de parkings privés disposent de places inoccupées qui restent inexploitées. Cette situation engendre une perte de temps, une augmentation de la consommation de carburant ainsi qu'une congestion accrue du trafic urbain.

Dans ce contexte, les technologies mobiles et les systèmes intelligents offrent de nouvelles opportunités pour améliorer la gestion du stationnement. En permettant la centralisation des informations, la réservation des places et l'automatisation des paiements, ces solutions contribuent à simplifier l'expérience des utilisateurs tout en optimisant l'exploitation des espaces de stationnement.

Ce projet de fin d'études porte sur la conception et le développement de **TuniPark**, une plateforme intelligente de gestion de parkings composée d'une application mobile destinée aux utilisateurs et d'une interface web réservée aux administrateurs. L'objectif principal du projet est de faciliter la recherche de places de stationnement publiques et privées, de permettre aux propriétaires de proposer leurs propres parkings, de gérer les sessions de stationnement et d'effectuer les paiements de manière sécurisée.

La solution développée intègre également un module d'IA permettant de recommander les parkings les plus pertinents en fonction de plusieurs critères, notamment la distance, le taux de disponibilité, les interactions des utilisateurs ainsi que les caractéristiques tarifaires. Cette approche tend à rendre l'expérience utilisateur plus fluide en proposant des recommandations personnalisées.

D'un point de vue technique, l'application mobile a été développée avec **Flutter**, tandis que l'interface d'administration a été réalisée avec **Angular**. Le Backend repose sur le framework **NestJS** et utilise **PostgreSQL** comme système de gestion de base de données via l'ORM **Prisma**. Les images sont stockées sur **Cloudinary**, les notifications Push sont assurées par **Firestore Cloud Messaging (FCM)**, les courriers électroniques sont envoyés à l'aide de **Nodemailer** et les paiements sont réalisés via la plateforme **Flouci**. Un microservice d'IA développé en **Python** avec

FastAPI exploite un modèle **Random Forest** afin de générer les recommandations de parkings.

Ce mémoire présente les différentes étapes de réalisation de ce projet. Il commence par une présentation du contexte général et de l'étude de l'existant, puis décrit l'analyse des besoins, l'architecture du système et la conception. Il présente ensuite l'implémentation des différentes fonctionnalités ainsi que les tests réalisés avant de se conclure par une synthèse des résultats obtenus et des perspectives d'amélioration.

Chapitre 1

Contexte Général et Étude Préalable

1.1 Introduction

Ce chapitre introductif présente le cadre général dans lequel ce projet de fin d'études a été réalisé. Il décrit dans un premier temps l'organisme d'accueil ainsi que son contexte professionnel, avant de présenter le cadre général du projet **RedSys**, en détaillant son activité, son domaine d'expertise ainsi que son environnement de travail.

Ensuite, le chapitre présente le contexte du projet **TuniPark**, une application conçue pour la gestion des places de stationnement. Il souligne les enjeux auxquels sont confrontés les conducteurs lorsqu'ils cherchent une place pour se garer, ainsi que les besoins qui ont conduit à l'élaboration de cette solution. Pour conclure, ce chapitre introduit la proposition de solution, les buts du projet et la méthode suivie pour réussir les diverses étapes de la conception, du développement et de la validation de l'application.

1.2 Entreprise d'accueil

1.2.1 Aperçu de RedSys

RedSys est une entreprise tunisienne spécialisée dans les technologies de l'information. Elle intervient principalement dans les domaines des infrastructures informatiques, des réseaux, du cloud et de la cybersécurité. L'entreprise a été créée par des professionnels du secteur avec l'objectif de proposer des solutions informatiques fiables, sécurisées et répondant aux besoins de ses clients.



FIGURE 1.1 – Logo de RedSys

RedSys accompagne les entreprises dans l'intégration, le déploiement et la maintenance de leurs systèmes informatiques. Ses services concernent notamment la mise en place d'infrastructures, l'intégration de solutions réseau, la migration vers le cloud ainsi que la sécurisation des environnements numériques.

L'entreprise attache également une grande importance à la qualité de ses services, à la réactivité de ses équipes et à l'évolution des compétences de ses collaborateurs. Cette approche lui permet de proposer des solutions évolutives et de construire des relations durables avec ses clients.

1.2.2 Domaines d'activité de RedSys

RedSys accompagne les entreprises dans la conception, le déploiement et la maintenance de solutions informatiques adaptées à leurs besoins. Son activité couvre plusieurs domaines, notamment l'administration des infrastructures informatiques, les réseaux, les services cloud, la cybersécurité ainsi que le développement de solutions numériques.

L'entreprise intervient auprès de clients issus de différents secteurs en proposant des services de conseil, d'intégration et de support technique. Elle met également l'accent sur l'innovation et l'utilisation de technologies modernes afin de répondre aux nouveaux besoins liés à la transformation numérique.

Grâce à son expertise technique et à son approche orientée qualité, RedSys développe des solutions fiables et évolutives, tout en assurant un accompagnement continu de ses clients.

1.2.3 Contexte organisationnel

Dans le cadre de sa stratégie de développement, RedSys encourage la réalisation de projets innovants répondant à des problématiques concrètes. C'est dans cette optique qu'a été proposé le projet **TuniPark**, une application mobile et une plateforme web destinées à faciliter la gestion et la réservation des places de parking.

Ce projet s'inscrit dans une démarche de transformation numérique visant à optimiser l'expérience des utilisateurs en proposant une solution simple, sécurisée et accessible. Il répond également aux besoins des gestionnaires de parkings en leur

offrant des outils permettant de suivre et d'administrer efficacement leurs espaces de stationnement.

1.3 Cadre général du projet

Le stationnement représente aujourd'hui un défi quotidien pour de nombreux conducteurs, notamment dans les zones urbaines où le nombre de places disponibles est souvent insuffisant. Cette situation engendre une perte de temps, une augmentation de la circulation ainsi qu'une consommation supplémentaire de carburant.

En parallèle, de nombreux espaces de stationnement privés restent inoccupés durant une partie de la journée. Ces espaces pourraient être exploités afin de répondre aux besoins des conducteurs tout en offrant une source de revenus complémentaire à leurs propriétaires.

Afin de répondre à cette problématique, **RedSys** a lancé le projet **TuniPark**. Dans le cadre de ce projet de fin d'études, une application mobile accompagnée d'une interface web d'administration a été conçue et développée afin de faciliter l'accès aux places de stationnement disponibles et de permettre aux propriétaires de proposer leurs espaces.

1.4 Problématique

Malgré l'existence de plusieurs solutions de stationnement, celles-ci ne répondent pas toujours aux besoins des conducteurs ni à ceux des propriétaires d'espaces privés. Les conducteurs rencontrent encore des difficultés pour trouver rapidement une place disponible, tandis que de nombreux propriétaires ne disposent pas d'un moyen simple et fiable pour proposer leurs espaces de stationnement.

La problématique de ce projet peut ainsi être formulée comme suit :

Comment développer une application permettant de faciliter l'accès aux places de stationnement disponibles tout en offrant aux propriétaires un moyen simple et sécurisé de proposer leurs espaces ?

1.5 Étude de l'existant

Plusieurs applications permettent aujourd'hui de rechercher des parkings, de consulter leur localisation et, dans certains cas, d'effectuer un paiement en ligne. La plupart de ces solutions sont principalement orientées vers les parkings publics ou les parkings gérés par des partenaires professionnels.

En revanche, elles offrent rarement aux particuliers la possibilité de proposer facilement leurs propres espaces de stationnement. Cette limitation réduit le nombre de places accessibles et ne permet pas d'exploiter pleinement les espaces privés disponibles.

1.6 Critique de l'existant

L'étude des solutions existantes met en évidence plusieurs limites. D'une part, l'offre de stationnement reste insuffisante dans certaines zones fortement fréquentées. D'autre part, les propriétaires d'espaces privés disposent de peu d'outils leur permettant de publier et de gérer simplement leurs annonces.

Ces limites justifient le développement d'une solution offrant à la fois une expérience simple pour les conducteurs et un moyen efficace pour les propriétaires de mettre leurs espaces de stationnement à disposition.

1.7 Solution proposée

Afin de répondre aux limites identifiées lors de l'étude de l'existant, une plateforme nommée **TuniPark** a été conçue et développée. Elle se compose d'une application mobile destinée aux conducteurs et aux propriétaires de parkings privés, ainsi que d'une interface web dédiée à l'administration de la plateforme.



FIGURE 1.2 – Icône de l'application TuniPark

L'application mobile offre aux conducteurs la possibilité de localiser des parkings, de consulter leurs informations, d'effectuer un paiement en ligne et de gérer leurs sessions de stationnement. Les propriétaires disposent d'un espace leur permettant de publier leurs parkings, de définir les tarifs, les horaires d'ouverture, la capacité d'accueil et de suivre les revenus générés par leurs espaces.

L'interface web est réservée à l'administrateur de la plateforme. Elle permet d'examiner les demandes de création de parkings, de valider ou de refuser les annonces proposées et d'assurer le suivi global du fonctionnement de l'application.

D'un point de vue technique, l'application mobile a été développée avec **Flutter**, tandis que le backend repose sur le framework **NestJS**. Les données sont gérées à l'aide de **PostgreSQL** via l'ORM **Prisma**. La visualisation cartographique est assurée par **Mapbox** et les transactions de paiement sont réalisées grâce à la plateforme **Flouci**.

1.8 Méthodologie de développement

1.8.1 Méthodologie Agile

Le développement de **TuniPark** a été réalisé en adoptant la méthodologie Agile [1]. Cette approche permet de développer l'application de manière progressive tout en intégrant les retours obtenus au cours du projet.

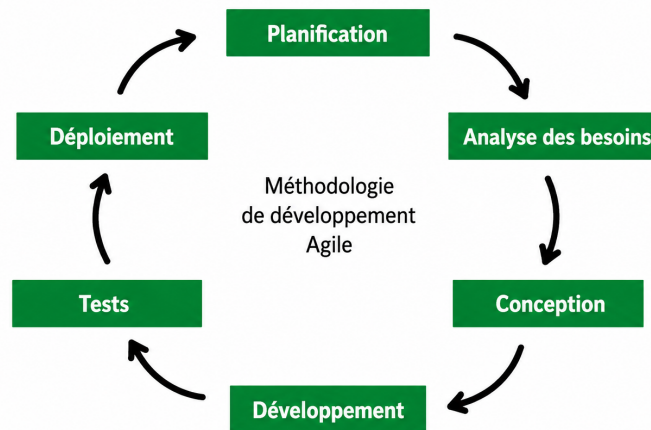


FIGURE 1.3 – Cycle de développement Agile

Comme illustré dans la Figure 1.3, le développement s'est déroulé selon plusieurs phases successives : l'identification des besoins, la conception à l'aide d'UML et de l'architecture, la réalisation de l'application mobile sous Flutter, le développement du backend avec NestJS, les tests et la validation, puis le déploiement.

Cette organisation itérative a permis de développer progressivement les différentes fonctionnalités de l'application, de corriger rapidement les anomalies détectées et d'améliorer continuellement la solution.

Les principaux avantages de cette approche sont :

- développement progressif de l'application ;

-
- adaptation aux évolutions des besoins ;
 - validation régulière des fonctionnalités développées ;
 - amélioration continue de la qualité de l'application.

1.8.2 Framework SCRUM

Afin d'organiser le développement, le framework SCRUM a été adopté [2]. Les fonctionnalités de l'application ont été regroupées dans un *Product Backlog* puis développées progressivement selon leur priorité.

Les premières itérations ont porté sur les fonctionnalités essentielles comme l'authentification, la gestion des utilisateurs et la recherche de parkings. Les itérations suivantes ont permis d'intégrer la gestion des parkings privés, les sessions de stationnements, les paiements, les notifications ainsi que le système de recommandation basé sur l'IA.

À la fin de chaque itération, les fonctionnalités développées ont été testées et validées avant de procéder à la suivante. Cette organisation a permis de garantir une progression structurée du projet tout en assurant la qualité de la solution développée.

1.9 Conclusion

Dans ce chapitre, nous avons présenté le cadre général du TuniPark, la problématique étudiée, les solutions existantes ainsi que leurs principales limites. Nous avons ensuite proposé la solution retenue et présenté la méthodologie Agile ainsi que le framework SCRUM adoptés pour organiser le développement du projet.

Le chapitre suivant est consacré à l'analyse et à la spécification des besoins. Il présente les acteurs du système, les besoins fonctionnels et non fonctionnels ainsi que les principaux diagrammes UML utilisés pour la conception de l'application.

Chapitre 2

Analyse et Spécification des Besoins

2.1 Introduction

Ce chapitre se concentre sur l'analyse et la spécification des besoins de l'application **TuniPark**, après avoir introduit le contexte global du projet ainsi que la solution suggérée. Cette phase consiste à repérer les différents intervenants du système, de définir les fonctionnalités attendues ainsi que les contraintes techniques et qualitatives à respecter au cours du développement.

Dans un premier temps, ce chapitre introduit les intervenants dans le système. Il énonce ensuite les exigences fonctionnelles et non fonctionnelles de l'application avant de présenter les diagrammes UML majeurs employés pour la modélisation du système.

2.2 Identification des acteurs

L'application TuniPark fait intervenir deux principaux acteurs : l'utilisateur et l'administrateur. Chacun possède des responsabilités spécifiques au sein du système.

2.2.1 Utilisateur

L'utilisateur représente toute personne utilisant l'application mobile. Selon son besoin, il peut agir en tant que conducteur recherchant une place de stationnement ou en tant que propriétaire souhaitant proposer un parking privé. Il peut rechercher des parkings, effectuer un paiement, démarrer une session de stationnement, publier un parking privé, consulter son historique ainsi que gérer son profil.

2.2.2 Administrateur

L'administrateur interagit l'interface web afin de superviser le fonctionnement de l'application. Il est chargé de consulter les demandes de création de parkings privés, d'approuver ou de rejeter les annonces, de gérer les parkings validés et de consulter les statistiques générales de la plateforme.

2.3 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que doit fournir l'application TuniPark.

L'application doit permettre à l'utilisateur de :

- créer un compte et s'authentifier ;
- se connecter avec Google ;
- réinitialiser son mot de passe ;
- rechercher des parkings à proximité ;
- consulter les parkings sur une carte interactive ;
- afficher les informations détaillées d'un parking ;
- sélectionner le nombre de places souhaitées ;
- effectuer un paiement sécurisé via Flouci ;
- démarrer automatiquement une session de stationnement après le paiement ;
- prolonger une session de stationnement ;
- consulter l'historique des sessions ;
- recevoir des notifications ;
- publier un parking privé ;
- ajouter des photos, les informations du parking ainsi qu'un document justificatif de propriété ;
- définir les tarifs et les horaires d'ouverture ;
- modifier ou archiver un parking publié ;
- consulter les revenus générés par ses parkings ;
- gérer son profil personnel.

L'application doit permettre à l'administrateur de :

- s'authentifier de manière sécurisée ;
- consulter les demandes de création de parkings privés ;

-
- approuver ou rejeter les parkings proposés ;
 - gérer les parkings validés ;
 - consulter les revenus et les statistiques globales ;
 - superviser le bon fonctionnement de la plateforme.

2.4 Besoins non fonctionnels

Les besoins non fonctionnels définissent les exigences de qualité que doit respecter l'application.

- **Sécurité** : Les données des utilisateurs doivent être protégées grâce à un mécanisme d'authentification sécurisé basé sur les JSON Web Tokens (JWT). Les sessions doivent être gérées de manière sécurisée à l'aide d'un Access Token et d'un Refresh Token. Les données sensibles stockées localement doivent être protégées et supprimées automatiquement lors de l'expiration de la session. Les paiements sont réalisés via la plateforme sécurisée Flouci.
- **Performance** : L'accès rapide aux fonctionnalités telles que la recherche de places de parking, la visualisation de la carte, les suggestions et les actions de réservation et de paiement est essentiel pour garantir une expérience utilisateur sans accroc.
- **Fiabilité** : Il est impératif que le système assure une disponibilité élevée des services et assure l'intégrité des données relatives aux utilisateurs, aux parkings, aux réservations et aux paiements.
- **Utilisabilité** : L'interface web d'administration et L'application mobile doivent offrir une interface intuitive, ergonomique et facile à utiliser.
- **Scalabilité** : L'architecture doit permettre l'ajout de nouveaux utilisateurs, de nouveaux parkings ainsi que de fonctionnalités émergentes sans dégrader les performances du système.
- **Maintenabilité** : Le code source doit être organisé de manière modulaire et respecter une architecture claire afin de faciliter la maintenance, les corrections et l'évolution de la plateforme.
- **Compatibilité** : L'application mobile faut être compatible avec les systèmes Android et iOS, tandis que l'interface d'administration doit être accessible depuis les navigateurs web récents les plus courants.

2.5 Diagramme global des cas d'utilisation

Le diagramme global des cas d'utilisation présente une vue d'ensemble des principales interactions entre les acteurs et l'application **TuniPark**. Il met en

évidence les principales fonctionnalités offertes par le système ainsi que les services accessibles à chaque acteur.

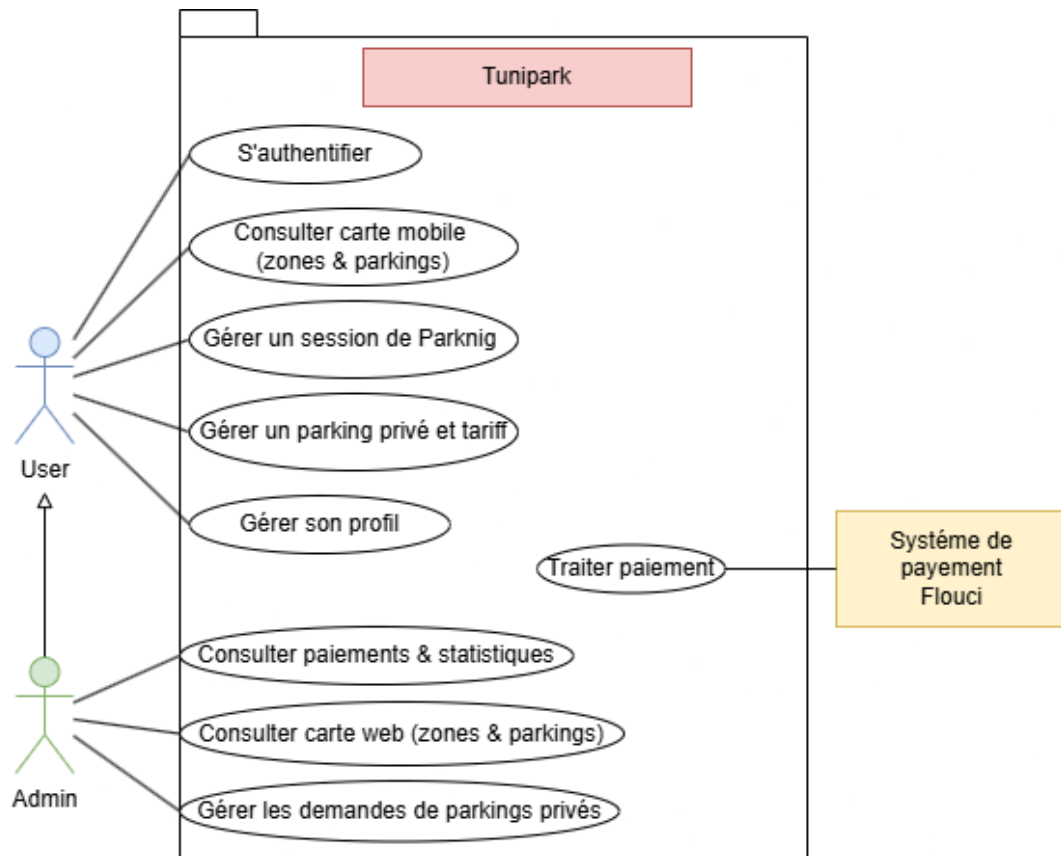


FIGURE 2.1 – Diagramme global des cas d'utilisation de l'application Tunipark

Comme l'illustre la Figure 2.1, l'utilisateur interagit avec l'application mobile afin de s'authentifier, consulter les parkings sur la carte, gérer ses sessions de stationnement, publier et administrer ses parkings privés ainsi que gérer son profil. Les opérations de paiement sont assurées par le service externe **Flouci**, intégré à l'application pour le traitement des transactions.

L'administrateur interagit l'interface web pour consulter la carte des parkings, gérer les demandes de création de parkings privés et accéder aux paiements ainsi qu'aux statistiques de la plateforme. Cette organisation permet de répartir clairement les responsabilités entre les différents acteurs tout en une gestion efficace du système.

2.6 Description des principaux cas d'utilisation

Cette section présente les principaux cas d'utilisation de l'application TuniPark. Les cas sélectionnés correspondent aux fonctionnalités essentielles proposées aux utilisateurs comme aux administrateurs, notamment la gestion des sessions de stationnement, la gestion des parkings privés, les paiements ainsi que l'administration de la plateforme.

2.6.1 Authentification

Cas d'utilisation	Gérer l'authentification
Rôle concerné	Utilisateur, Administrateur
Objectif	Permettre à l'utilisateur ou à l'administrateur d'accéder à son espace personnel de manière sécurisée.
Précondition	L'utilisateur possède déjà un compte enregistré dans le système.
Scénario principal	L'utilisateur renseigne son adresse e-mail et son mot de passe. Le système vérifie les informations puis autorise l'accès à l'application selon le rôle de l'utilisateur.
Exception	Si les informations d'authentification sont incorrectes, Le système présente un message d'erreur et nie l'accès.
Postcondition	L'utilisateur est connecté et peut accéder aux fonctionnalités autorisées.

TABLE 2.1 – Authentification

2.6.2 Gestion d'une session de parking

Cas d'utilisation	Gérer une session de parking
Rôle concerné	Utilisateur
Objectif	Permettre à l'utilisateur de démarrer, consulter et arrêter une session de stationnement.
Précondition	L'utilisateur est authentifié et un parking est disponible.
Scénario principal	L'utilisateur sélectionne un parking, consulte ses informations, effectue le paiement puis le système démarre automatiquement la session de stationnement.
Exception	Si aucune place n'est disponible ou si une erreur survient, le système notifie l'utilisateur.
Postcondition	Une session de stationnement est créée et enregistrée.

TABLE 2.2 – Gestion d'une session de parking

2.6.3 Gestion d'un parking privé

Cas d'utilisation	Gérer un parking privé
Rôle concerné	Utilisateur
Objectif	Permettre à un utilisateur de publier et gérer un parking privé ainsi que sa tarification.
Précondition	L'utilisateur est authentifié.
Scénario principal	L'utilisateur renseigne les informations du parking (adresse, nombre de places, disponibilité et tarif), puis soumet sa demande. Après validation par l'administrateur, le parking devient disponible sur la plateforme.
Exception	Si les informations sont incomplètes ou invalides, la demande est refusée jusqu'à correction.
Postcondition	Le parking est enregistré ou mis à jour dans le système.

TABLE 2.3 – Gestion d'un parking privé

2.6.4 Paiement d'une session

Cas d'utilisation	Effectuer un paiement
Rôle concerné	Utilisateur
Objectif	Permettre à l'utilisateur de payer les frais de stationnement via le service Flouci.
Précondition	L'utilisateur a sélectionné un parking.
Scénario principal	Le système calcule le montant à payer selon les informations du parking. L'utilisateur est redirigé vers Flouci afin d'effectuer le paiement. Après validation de la transaction, la session de stationnement est créée automatiquement.
Exception	Si le paiement échoue ou est annulé, la transaction est marquée comme non effectuée et l'utilisateur est informé.
Postcondition	Le paiement est enregistré et la session de stationnement est démarrée.

TABLE 2.4 – Paiement d'une session

2.6.5 Validation des parkings privés

Cas d'utilisation	Traiter les demandes de parkings privés
Rôle concerné	Administrateur
Objectif	Permettre à l'administrateur de valider ou refuser les requêtes de création de parkings privés.
Précondition	Une ou plusieurs demandes sont en attente de validation.
Scénario principal	L'administrateur consulte les demandes, vérifie les informations fournies puis accepte ou refuse la publication du parking.
Exception	Si les informations sont incomplètes ou incorrectes, la demande est rejetée avec un motif.
Postcondition	Le statut de la demande est mis à jour et l'utilisateur est informé de la décision.

TABLE 2.5 – Validation des parkings privés

2.7 Product Backlog

Ce dernier regroupe les principales fonctionnalités identifiées lors de l'analyse des besoins de l'application **TuniPark**. Il a servi de référence tout au long du développement afin de planifier les tâches, définir les priorités et organiser les différentes itérations du projet.

Les fonctionnalités ont été réparties selon plusieurs modules, notamment l'authentification, la gestion des parkings, les sessions de stationnement, les paiements, les notifications ainsi que l'administration de l'application.

TABLE 2.6 – Product Backlog de l'application TuniPark

ID	Module	User Story	Acteur	Priorité
1	Authentification	En tant que personne utilisant l'application, mon désir est de constituer un compte pour avoir accès aux services proposés.	Utilisateur	Élevée

ID	Module	User Story	Acteur	Priorité
2	Authentification	En tant qu'utilisateur, mon souhait est de me connecter en toute sécurité pour accéder à mon espace personnel.	Utilisateur / Admin	Élevée
3	Authentification	Dans mon rôle d'utilisateur, je désire ré-initialiser mon mot de passe via un code OTP reçu par e-mail ou téléphone.	Utilisateur	Élevée
4	Carte	Dans mon rôle d'utilisateur, je désire visualiser les parkings disponibles sur une carte interactive afin de choisir celui qui me convient.	Utilisateur	Élevée
5	Recherche	Dans mon rôle d'utilisateur, je désire rechercher un parking afin de trouver rapidement une place disponible.	Utilisateur	Élevée
6	Session	Dans mon rôle d'utilisateur, je désire démarrer une session de stationnement après le paiement.	Utilisateur	Élevée
7	Paielement	Dans mon rôle d'utilisateur, je désire payer ma session de stationnement via Flouci.	Utilisateur	Élevée
8	Historique	Je souhaite, en tant qu'utilisateur, avoir la possibilité de consulter l'historique de mes sessions de stationnement.	Utilisateur	Moyenne
9	Notifications	Dans mon rôle d'utilisateur, je désire recevoir des notifications concernant mes sessions et mes parkings.	Utilisateur	Moyenne
10	Parking privé	Dans mon rôle d'utilisateur, je désire publier un parking privé afin de le proposer à la location.	Utilisateur	Élevée
11	Parking privé	Dans mon rôle d'utilisateur, je désire ajouter des photos, une description, un tarif, des horaires et un document justificatif lors de la création d'un parking.	Utilisateur	Élevée
12	Parking privé	Dans mon rôle d'utilisateur, je désire modifier ou archiver mes parkings.	Utilisateur	Moyenne
13	Revenus	Dans mon rôle d'utilisateur, je désire consulter les revenus générés par mes parkings.	Utilisateur	Moyenne

ID	Module	User Story	Acteur	Priorité
14	Administration	En tant qu'administrateur, je souhaite consulter les demandes de création de parkings privés.	Admin	Élevée
15	Administration	En tant qu'administrateur, je souhaite approuver ou rejeter les parkings proposés.	Admin	Élevée
16	Tableau de bord	En tant qu'administrateur, je souhaite consulter les statistiques générales de l'application.	Admin	Moyenne
17	Revenus	En tant qu'administrateur, je souhaite consulter les revenus générés par la plateforme.	Admin	Moyenne

Ce Product Backlog a permis de structurer le développement de TuniPark en définissant les fonctionnalités à implémenter selon leur priorité. Les fonctionnalités critiques, notamment l'authentification, la gestion des parkings, les sessions de stationnement et les paiements, ont été développées en priorité. Les fonctionnalités complémentaires, comme les notifications, les statistiques et la gestion des revenus, ont ensuite été intégrées afin d'enrichir l'application et d'améliorer l'expérience utilisateur.

2.8 Conclusion

Ce chapitre a présenté l'analyse et la spécification des besoins de l'application **TuniPark**. Il a permis d'identifier les différents acteurs du système, de définir les besoins fonctionnels et non fonctionnels, ainsi que de présenter le diagramme global des cas d'utilisation.

Les principaux cas d'utilisation ont ensuite été décrits afin de mieux comprendre les interactions entre les utilisateurs, l'administrateur et l'application. Enfin, le Product Backlog a été établi pour organiser les fonctionnalités à développer et définir leurs priorités.

Le prochain chapitre portera sur la conception et à l'architecture de l'application. Il présentera l'architecture générale du système, les choix techniques adoptés, la modélisation de la base de données ainsi que les différents diagrammes UML utilisés lors de la conception.

Chapitre 3

Conception et Architecture du Système

3.1 Introduction

Après avoir examiné les exigences fonctionnelles et non fonctionnelles de **TuniPark**, cette étape consiste à définir l'architecture du système ainsi que les choix techniques retenus pour son développement. L'objectif est de concevoir une architecture modulaire, évolutive et facilement maintenable, capable de répondre aux différents besoins de l'application.

Ce chapitre présente tout d'abord l'architecture générale du système, puis décrit les architectures logique et physique. Les principales technologies utilisées sont ensuite présentées avant d'introduire la conception de la base de données ainsi que les principaux diagrammes UML réalisés durant la phase de conception.

3.2 Architecture du système

L'application **TuniPark** repose sur une architecture client–serveur composée d'une application mobile destinée aux utilisateurs, d'une interface web réservée à l'administrateur, d'un serveur Backend, d'un microservice d'IA ainsi que d'une base de données relationnelle. Cette architecture assure une séparation claire entre les différentes couches du système afin d'améliorer la maintenabilité, la sécurité et l'évolutivité de l'application.

3.2.1 Architecture logique

Cette partie décrit les principaux composants logiciels de l'application ainsi que leurs interactions. Le système est organisé en cinq couches : la couche de

présentation, la couche métier, la couche d'accès aux données, la couche de données et la couche des services externes.

La couche de présentation comprend l'application mobile développée avec **Flutter** [3], destinée aux utilisateurs, ainsi que l'interface web d'administration développée avec **Angular** [4]. L'application mobile permet notamment de rechercher des parkings, gérer des sessions de stationnement, publier des parkings privés, effectuer des paiements, consulter les statistiques personnelles et recevoir des notifications. L'interface web permet à l'administrateur de superviser les utilisateurs, les parkings, les paiements et les différentes informations de la plateforme.

La couche métier est implémentée à l'aide du framework **NestJS** [5]. Elle expose une API REST sécurisée et regroupe l'ensemble de la logique métier de l'application. Cette couche est composée de plusieurs modules, notamment l'authentification, la gestion des utilisateurs, des parkings, des sessions de stationnement, des parkings privés, des paiements, des notifications, des statistiques ainsi que du système de recommandation basé sur l'IA.

La couche d'accès aux données repose sur **Prisma ORM** [6]. Celui-ci facilite les échanges entre le Backend et la base de données PostgreSQL tout en garantissant une manipulation fiable et sécurisée des données.

La couche de données fait appel à une base de données de type relationnel **PostgreSQL** [7]. Elle assure le stockage des informations relatives aux utilisateurs, aux parkings, aux sessions de stationnement, aux paiements, aux notifications ainsi qu'aux interactions exploitées par le système de recommandation.

Enfin, l'application s'appuie sur plusieurs services externes. Les images des parkings sont stockées sur **Cloudinary** [8], les notifications Push sont envoyées grâce à **Firestore Cloud Messaging (FCM)** [9], les paiements électroniques sont réalisés via **Flouci** [10], les courriers électroniques sont envoyés à l'aide de **Nodemailer** [11], tandis que l'envoi des codes OTP par SMS est prévu via **Twilio** [12]. Un microservice développé avec **FastAPI** [13] héberge également le modèle d'IA basé sur l'algorithme **Random Forest** [14], chargé de calculer les recommandations de parkings.

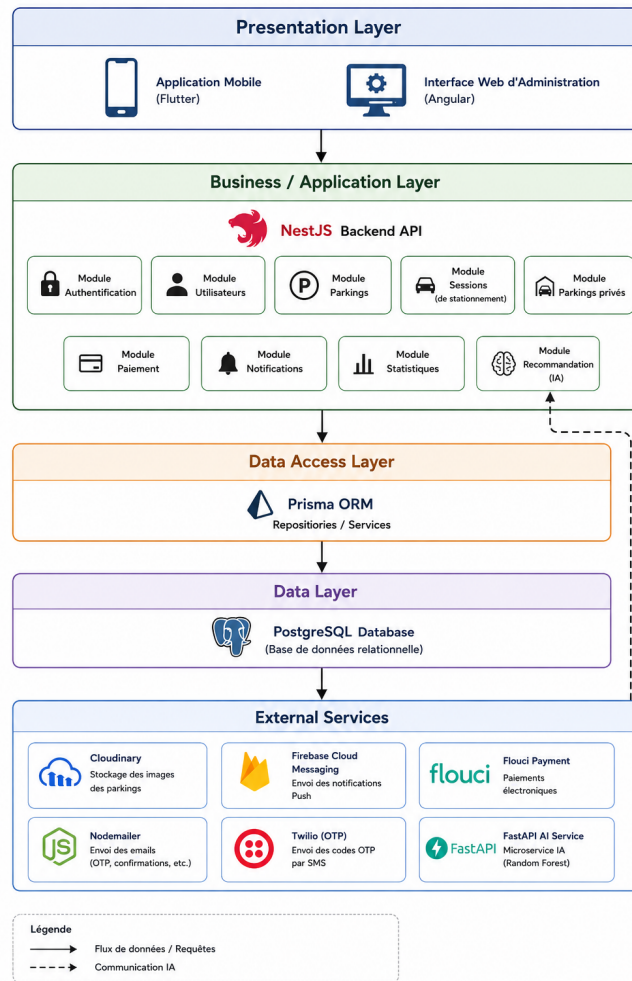


FIGURE 3.1 – Architecture logique de Tunipark

Comme l'illustre la Figure 3.1, les interfaces mobile et web communiquent avec le Backend via des requêtes REST sécurisées. Le Backend centralise la logique métier, interagit avec PostgreSQL par l'intermédiaire de Prisma ORM et communique avec les différents services externes tels que Cloudinary, Flouci, Firebase Cloud Messaging, Nodemailer ainsi que le microservice d'IA développé avec FastAPI.

3.2.2 Architecture physique

Cette architecture présente le déploiement réel des différents composants de l'application Tunipark. Le système est accessible à travers deux interfaces clientes : l'application mobile développée avec **Flutter** et l'interface web d'administration développée avec **Angular**.

L'interface web d'administration est hébergée sur **Vercel** [15], tandis que le Backend développé avec **NestJS**, le microservice d'IA développé avec **FastAPI** ainsi que la base de données **PostgreSQL** sont déployés sur **Render** [16]. Les interfaces clientes communiquent avec le Backend au moyen de requêtes HTTPS vers une API REST sécurisée.

Le Backend centralise la logique métier de l'application et assure la communication avec la base de données ainsi qu'avec les différents services externes. Les images des parkings sont stockées sur **Cloudinary**, les notifications Push sont envoyées via **Firebase Cloud Messaging**, les paiements électroniques sont traités par **Flouci** et les courriers électroniques sont envoyés à l'aide de **Nodemailer**.

Le service **Twilio** est prévu pour l'envoi des codes OTP par SMS, mais il n'est pas encore opérationnel dans la version actuelle de l'application. Le microservice FastAPI héberge le modèle de recommandation basé sur l'algorithme Random Forest et communique avec le Backend NestJS à travers des requêtes HTTP.

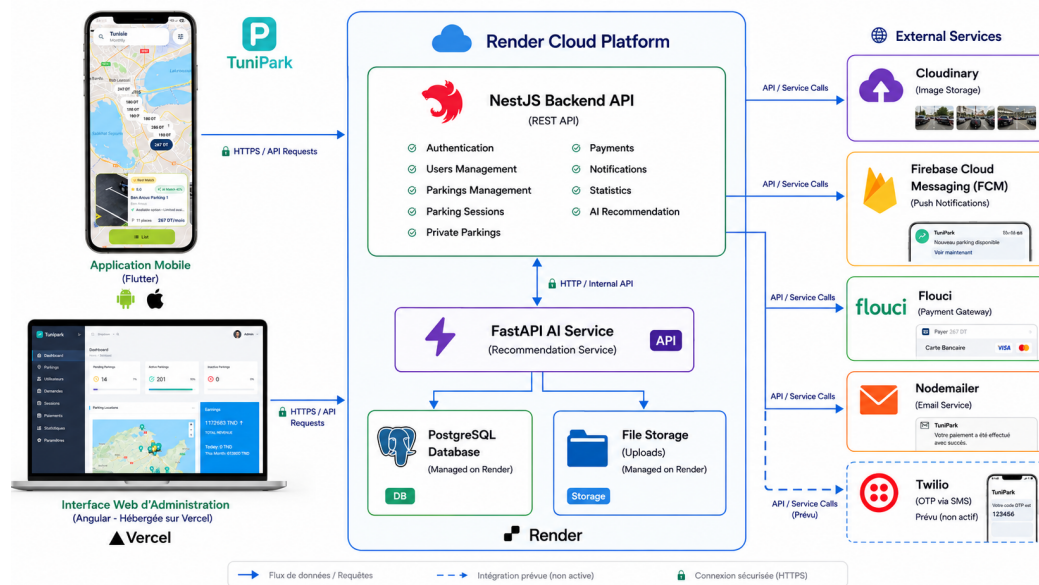


FIGURE 3.2 – Architecture physique de l'application Tunipark

Comme le montre la Figure 3.2, les différents composants sont déployés sur des services Cloud distincts. Cette séparation permet de simplifier le déploiement, de faciliter la maintenance et de faire évoluer indépendamment le Backend, l'interface web et le service d'IA.

3.3 Choix techniques

Les technologies retenues pour le développement de l'application **TuniPark** ont été sélectionnées en fonction des exigences du projet, notamment en matière de performance, de sécurité, de maintenabilité et d'évolutivité. L'objectif était de mettre en place une architecture moderne, reposant sur des outils fiables, afin de concevoir une application mobile, une interface web d'administration, un Backend sécurisé et une base de données performante, tout en assurant l'interfaçage avec des services externes et un système de recommandation basé sur l'IA.

3.3.1 Flutter

Flutter [3] a été choisi pour développer l'application mobile de TuniPark. Ce framework développé par Google permet de concevoir une application multiplateforme à partir d'une seule base de code, compatible avec Android et iOS.

L'application mobile permet aux utilisateurs de rechercher des parkings, consulter leur disponibilité sur une carte interactive, démarrer et arrêter une session de stationnement, effectuer des paiements, gérer leurs parkings privés ainsi que recevoir des notifications en temps réel.



FIGURE 3.3 – Logo du framework Flutter

3.3.2 Angular

Angular [4] a été utilisé pour développer l'interface web d'administration de TuniPark. Cette application permet à l'administrateur de superviser la plateforme, gérer les utilisateurs, valider les demandes de parkings privés, consulter les statistiques ainsi que suivre l'ensemble des opérations réalisées sur le système.

L'application Angular communique avec le Backend via des API REST sécurisées.



FIGURE 3.4 – Logo du framework Angular

3.3.3 NestJS

Le Backend de TuniPark a été développé avec NestJS [5]. Ce framework basé sur Node.js et TypeScript offre une architecture modulaire facilitant l'organisation des différents modules de l'application.

Il centralise la logique métier du système, notamment l'authentification, la gestion des utilisateurs, des parkings, des sessions de stationnement, des paiements, des notifications, ainsi que la communication avec le microservice d'IA.



FIGURE 3.5 – Logo du framework NestJS

3.3.4 PostgreSQL et Prisma

Les données de TuniPark sont stockées dans une base de données relationnelle PostgreSQL [7]. Cette solution garantit l'intégrité des données, une gestion fiable des liens entre les entités, ainsi que de bonnes performances lors des requêtes.

L'accès à la base de données est assuré par Prisma ORM [6], qui simplifie les opérations de création, de lecture, de modification et de suppression des données tout en offrant une sécurité de typage grâce à TypeScript.



FIGURE 3.6 – Logo de PostgreSQL

3.3.5 FastAPI et Random Forest

Pour offrir une meilleure expérience aux utilisateurs, TuniPark intègre un microservice d'IA développé avec FastAPI [13] en Python. Ce service expose des API REST consommées par le Backend NestJS.

Le modèle de recommandation repose sur l'algorithme Random Forest [14]. Il analyse plusieurs critères tels que la distance entre l'utilisateur et le parking, le taux de disponibilité, le taux de conversion des consultations en réservations, le taux de prolongation des sessions ainsi que le tarif du parking afin de calculer un score de recommandation.

Les parkings sont ensuite classés selon ce score afin de proposer en priorité les emplacements les plus pertinents pour chaque utilisateur.



FIGURE 3.7 – Microservice IA développé avec FastAPI

3.3.6 Authentification JWT

La sécurisation des accès repose sur JSON Web Token (JWT) [17]. Après une authentification réussie, le Backend génère un Access Token valide pendant une heure ainsi qu'un Refresh Token valide pendant trois jours.

Lorsque l'Access Token expire, un intercepteur Dio renouvelle automatiquement celui-ci à l'aide du Refresh Token sans intervention de l'utilisateur. Si le Refresh Token arrive également à expiration, la session est automatiquement fermée, les informations locales sont supprimées et l'utilisateur est orienté vers la page de connexion.



FIGURE 3.8 – Authentification basée sur JWT

3.3.7 Cloudinary

Cloudinary [8] est utilisé pour le stockage des images des parkings. Ce service Cloud permet d'héberger les images, de les optimiser automatiquement et de fournir des liens sécurisés pour leur affichage dans l'application.



FIGURE 3.9 – Logo de Cloudinary

3.3.8 Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) [9] permet l'envoi de notifications Push aux utilisateurs. Il est utilisé pour informer les utilisateurs des validations de leurs parkings, des paiements, des rappels de session ainsi que des différentes notifications générées par l'application.



FIGURE 3.10 – Logo de Firebase Cloud Messaging

3.3.9 Flouci

Flouci [10] est la solution de paiement électronique intégrée à TuniPark. Elle permet aux utilisateurs d'effectuer leurs paiements de manière sécurisée directement depuis l'application mobile avant le démarrage d'une session de stationnement.



FIGURE 3.11 – Logo de Flouci

3.3.10 Nodemailer

Nodemailer [11] est utilisé pour l'envoi automatique des courriers électroniques, notamment lors de la création d'un compte, de la changement de mot de passe et de l'envoi des codes OTP par courrier électronique.



FIGURE 3.12 – Logo de Nodemailer

3.4 Conception de la base de données

La base de données de l'application **TuniPark** repose sur le SGBD relationnelles **PostgreSQL** [7]. Elle est structurée autour de plusieurs tables correspondant aux

principaux domaines fonctionnels de l'application, notamment la gestion des utilisateurs, des parkings, des sessions de stationnement, des paiements, des notifications et des interactions utilisées par le système de recommandation.

L'accès aux données est assuré par **Prisma ORM** [6], qui facilite la définition des relations entre les entités et les opérations réalisées depuis le Backend NestJS. Cette organisation permet de préserver la cohérence des données et de faciliter l'évolution du système.

3.4.1 Tables de gestion des utilisateurs et des accès

Table	Rôle dans l'application
User	Sauvegarde les informations des utilisateurs, notamment leur identité, leurs coordonnées, leur mot de passe, leur rôle ainsi que l'état de vérification de leur adresse électronique et de leur numéro de téléphone. Cette table contient également les informations temporaires nécessaires à la vérification et à la changement de mot de passe par code OTP.
FcmToken	Stocke les jetons Firebase associés aux appareils des utilisateurs afin de permettre l'envoi de notifications Push. Un utilisateur peut posséder plusieurs jetons lorsqu'il utilise plusieurs appareils.

TABLE 3.1 – Tables de gestion des utilisateurs et des accès

3.4.2 Tables de gestion des parkings

Table	Rôle dans l'application
Parking	Stocke les informations principales d'un parking, notamment son titre, son type, sa description, son emplacement, ses horaires d'ouverture, ses photos, ses caractéristiques, le nombre total de places et le nombre de places disponibles. Elle conserve également son statut de validation ainsi que la référence de son propriétaire.
Tariff	Stocke les informations tarifaires associées à un parking, telles que le prix par unité, par jour ou par mois, ainsi que les durées minimale et maximale autorisées. Chaque parking possède au maximum un tarif.

TABLE 3.2 – Tables de gestion des parkings

3.4.3 Tables de gestion des sessions de stationnement

Table	Rôle dans l'application
ParkingSession	Stocke les sessions de stationnement ouvertes par les utilisateurs. Elle contient le parking sélectionné, les informations du véhicule, la date de début, la date de fin, la durée payée ainsi que le statut de la session.

TABLE 3.3 – Table de gestion des sessions de stationnement

3.4.4 Table de gestion des paiements

Table	Rôle dans l'application
Payment	Stocke les paiements associés aux sessions de stationnement. Elle conserve le fournisseur de paiement, le montant, le statut de la transaction, la référence retournée par le fournisseur ainsi que la date de confirmation du paiement.

TABLE 3.4 – Table de gestion des paiements

3.4.5 Tables de gestion des notifications

Table	Rôle dans l'application
Notification	Stocke les notifications destinées aux utilisateurs, notamment leur titre, leur contenu, leur type, les données complémentaires ainsi que leur état de lecture.
FcmToken	Associe chaque appareil à un jeton Firebase afin de transmettre les notifications Push au bon utilisateur.

TABLE 3.5 – Tables liées aux notifications

3.4.6 Table utilisée par le système de recommandation

Table	Rôle dans l'application
ParkingInteraction	Enregistre les interactions réalisées par les utilisateurs avec les parkings. Les interactions prises en charge sont la consultation d'un parking, le démarrage d'une session et la prolongation d'une session. Ces données permettent de calculer les taux de conversion et de prolongation exploités par le système de recommandation.

TABLE 3.6 – Table utilisée par le système de recommandation

3.4.7 Diagramme de classes

Le schéma de classes décrit la configuration globale de l'application **TuniPark**. Il souligne les entités clés du système, leurs caractéristiques ainsi que les liens qui les unissent. Le modèle couvre notamment la gestion des utilisateurs, des parkings, des sessions de stationnement, des paiements, des notifications et des interactions utilisées par le système de recommandation.

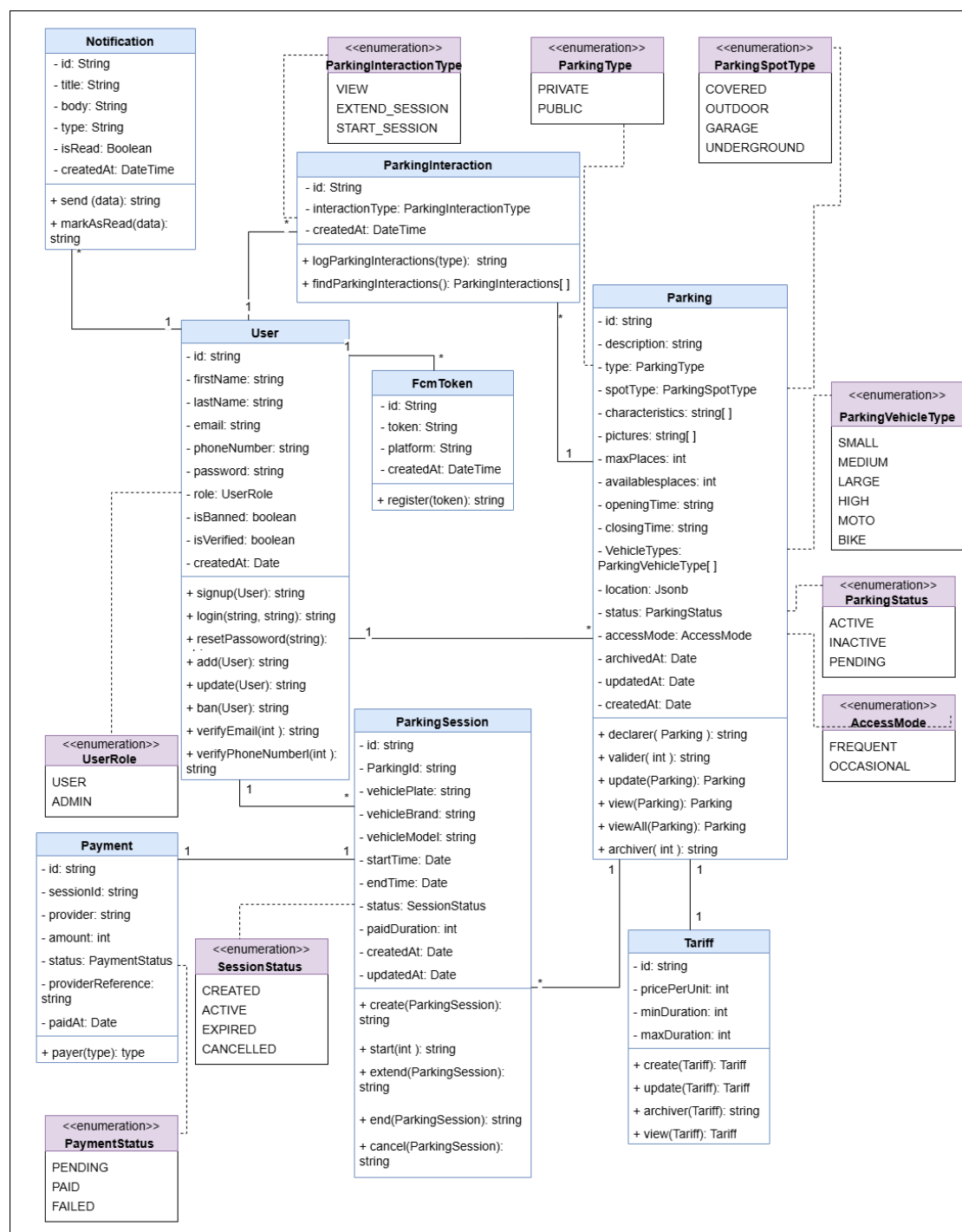


FIGURE 3.13 – Diagramme de classes de l'application TuniPark

Comme illustré dans la Figure 3.13, l'entité **User** occupe une place centrale dans

le système. Elle contient les informations personnelles et les données nécessaires à l'authentification, ainsi que le rôle attribué à chaque utilisateur. Un utilisateur peut posséder plusieurs parkings, ouvrir plusieurs sessions de stationnement, recevoir des notifications et générer différentes interactions avec les parkings.

L'entité **Parking** regroupe les informations descriptives d'un espace de stationnement, notamment son type, son emplacement, ses horaires d'ouverture, ses caractéristiques, ses images, sa capacité et son statut. Chaque parking appartient à un propriétaire et peut être associé à plusieurs sessions de stationnement et interactions. Une relation individuelle relie également chaque parking à son tarif, représenté par l'entité **Tariff**.

L'entité **ParkingSession** représente une session de stationnement ouverte par un utilisateur dans un parking donné. Elle contient les informations du véhicule, les dates de début et de fin, la durée payée ainsi que l'état de la session. Une session peut être associée à un seul paiement, stocké dans l'entité **Payment**, qui conserve le montant, le fournisseur, le statut de la transaction et sa référence externe.

Les entités **Notification** et **FcmToken** prennent en charge l'envoi et le suivi des notifications. Enfin, l'entité **ParkingInteraction** enregistre les actions réalisées sur les parkings, telles que la consultation, le démarrage ou la prolongation d'une session. Ces données sont exploitées pour calculer les indicateurs utilisés par le système de recommandation.

3.5 Diagrammes de séquence

Les diagrammes de séquence permettent de représenter le comportement dynamique de l'application TuniPark. Ils illustrent les échanges entre les différents acteurs, l'application mobile, l'interface web d'administration, le serveur Backend, la base de données ainsi que les services externes tels que Flouci, Firebase Cloud Messaging et le microservice d'IA.

3.5.1 Diagramme de séquence de l'authentification

Ce diagramme met en évidence la démarche d'authentification d'un utilisateur au sein de l'application **TuniPark**. Après avoir saisi son adresse électronique ou son numéro de téléphone ainsi que son mot de passe, l'application mobile envoie une requête d'authentification au Backend. Celui-ci vérifie l'existence de l'utilisateur dans la base de données, contrôle la validité des informations d'identification puis génère un *Access Token* et un *Refresh Token* [17]. Les jetons générés sont ensuite retournés à l'application et stockés de manière sécurisée sur l'appareil afin de permettre l'accès aux fonctionnalités protégées. En cas d'informations d'identification

invalides, un message d'erreur est renvoyé à l'utilisateur et l'authentification est refusée.

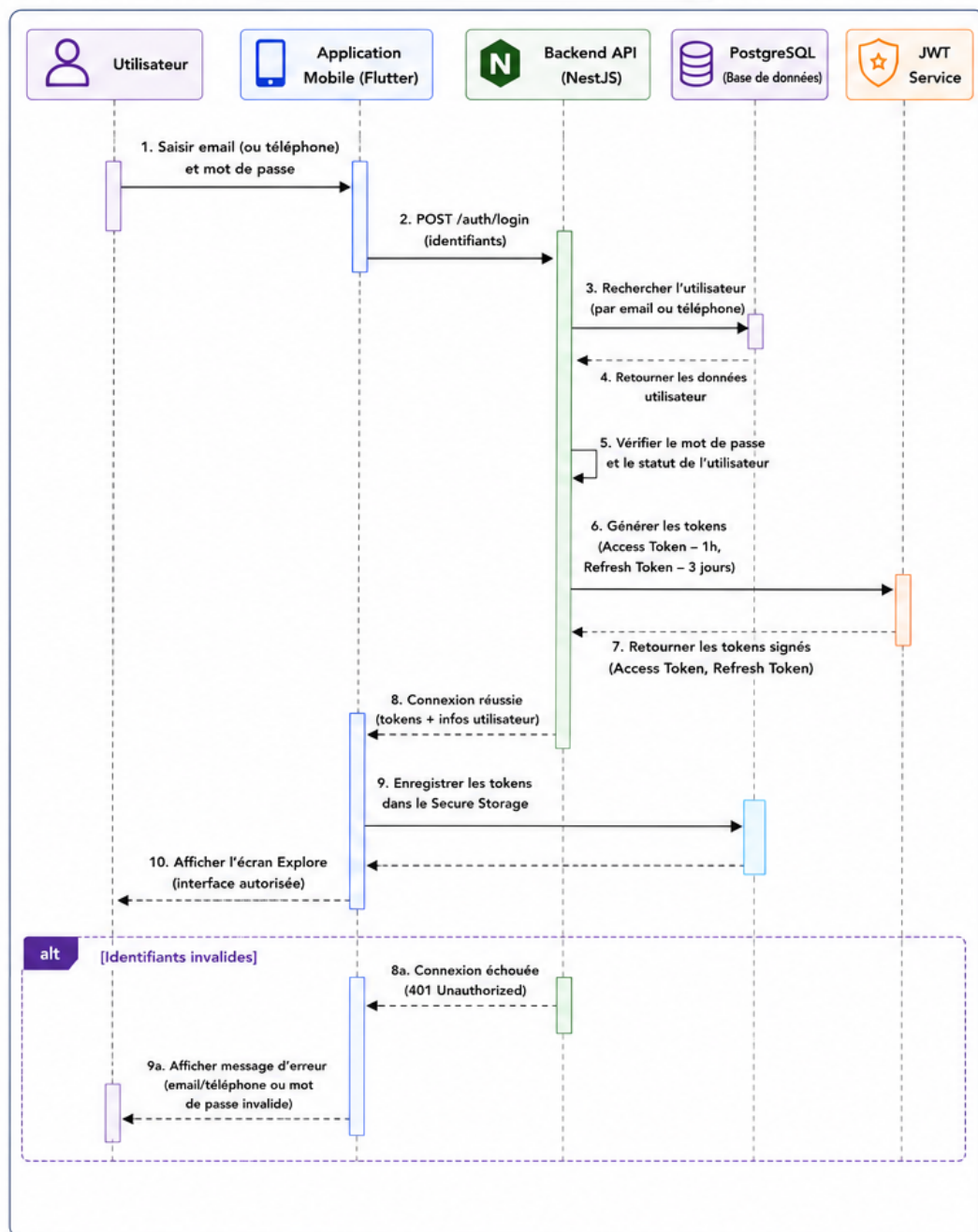


FIGURE 3.14 – Authentification

3.5.2 Diagramme de séquence du renouvellement automatique du jeton

Ce diagramme illustre le mécanisme de renouvellement automatique de l'*Access Token*. Lorsqu'une requête retourne une réponse indiquant l'expiration du jeton d'accès, l'application utilise le *Refresh Token* précédemment enregistré afin de demander un nouveau jeton au Backend. Après validation du *Refresh Token*, le Backend génère un nouvel *Access Token* qui est enregistré dans le stockage sécurisé de l'application. La requête initiale est ensuite relancée de manière transparente, sans intervention de l'utilisateur. Si le *Refresh Token* est invalide ou expiré, les jetons sont supprimés et l'utilisateur est automatiquement redirigé vers l'écran de connexion.

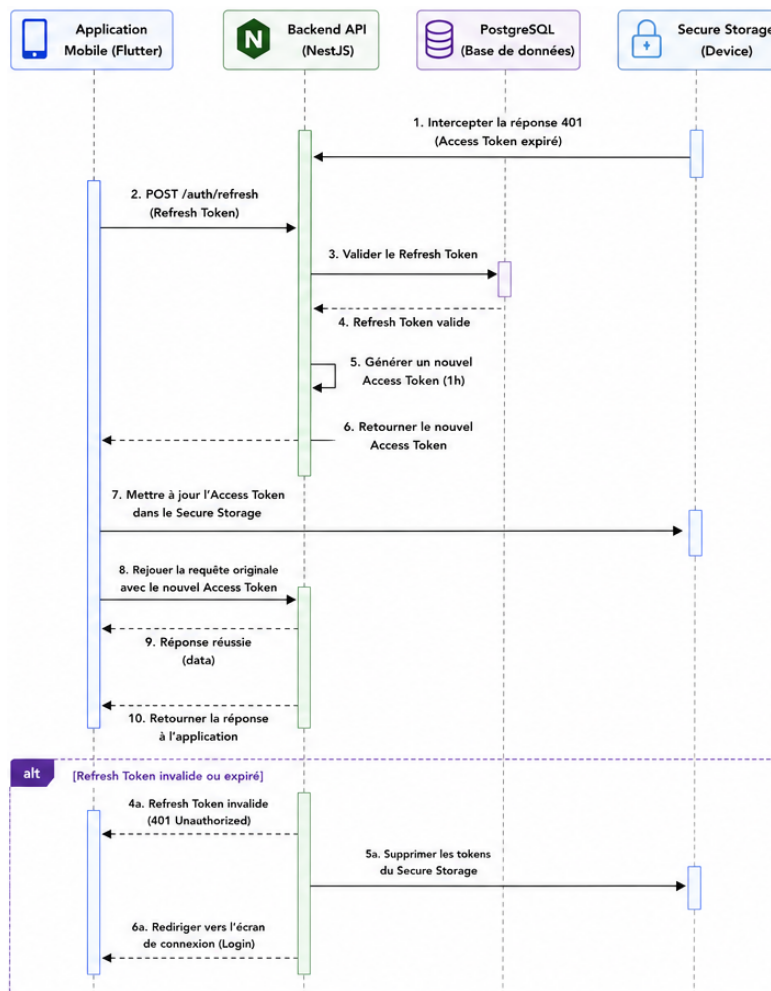


FIGURE 3.15 – Renouvellement automatique du jeton

3.5.3 Diagramme de séquence de la création d'une session de stationnement

Ce diagramme illustre le processus de création d'une session de stationnement dans l'application **TuniPark**. Après avoir sélectionné un parking et renseigné les informations nécessaires, telles que la durée souhaitée, le véhicule et le numéro d'immatriculation, l'utilisateur soumet une demande de création de session. Le Backend vérifie alors la disponibilité du parking dans la base de données, crée une nouvelle session avec le statut **CREATED** et enregistre les informations correspondantes. Les détails de la session sont ensuite retournés à l'application mobile afin d'afficher un récapitulatif avant le paiement.

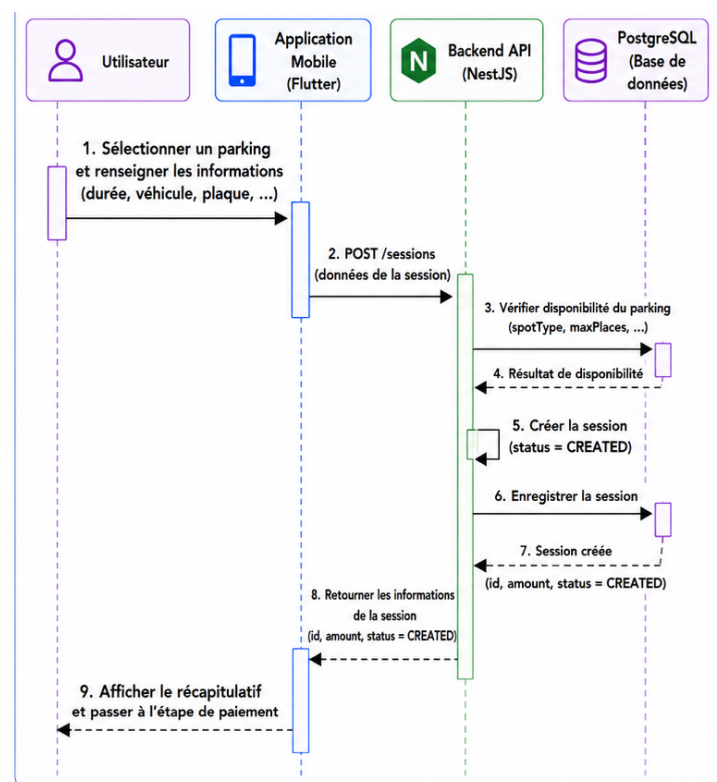


FIGURE 3.16 – Création d'une session de stationnement

3.5.4 Diagramme de séquence du paiement d'une session de stationnement

Ce diagramme présente le processus de paiement d'une session de stationnement à l'aide de la passerelle de paiement **Flouci** [10]. Après validation du récapitulatif,

l'utilisateur est renvoyé vers l'interface de paiement sécurisée de Flouci afin d'effectuer la transaction. Une fois le paiement terminé, Flouci transmet automatiquement le résultat au Backend via un *Webhook*. En cas de succès, le Backend met à jour l'état du paiement, active la session de stationnement en changeant son statut de **CREATED** à **ACTIVE** et envoie une notification de confirmation à l'utilisateur. Si le paiement échoue ou est annulé, le paiement est marqué comme échoué, la session conserve son statut **CREATED** et l'application informe l'utilisateur afin qu'il puisse réessayer ultérieurement.

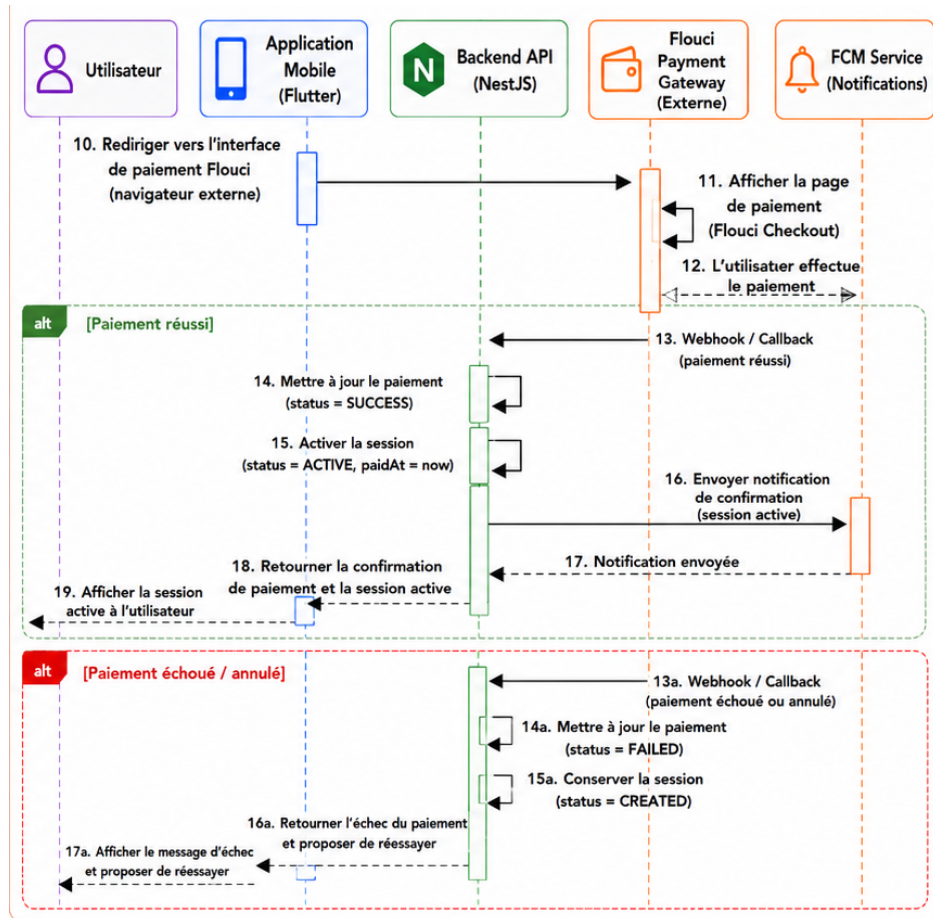


FIGURE 3.17 – Paiement d'une session de stationnement

3.5.5 Diagramme de séquence de la recommandation intelligente des parkings

Ce diagramme illustre le fonctionnement du système de recommandation des parkings. Après avoir obtenu la position de l'utilisateur, l'application envoie une

requête au Backend. Celui-ci récupère les parkings disponibles ainsi que leurs statistiques d'utilisation, puis transmet les caractéristiques de chaque parking au microservice d'IA développé avec FastAPI [13]. Le modèle Random Forest [14] calcule un score de recommandation pour chaque parking et retourne les résultats au Backend, qui les trie avant de les renvoyer à l'application mobile. En cas d'indisponibilité du microservice, le Backend applique automatiquement un algorithme de secours basé sur des règles afin d'assurer la continuité du service.

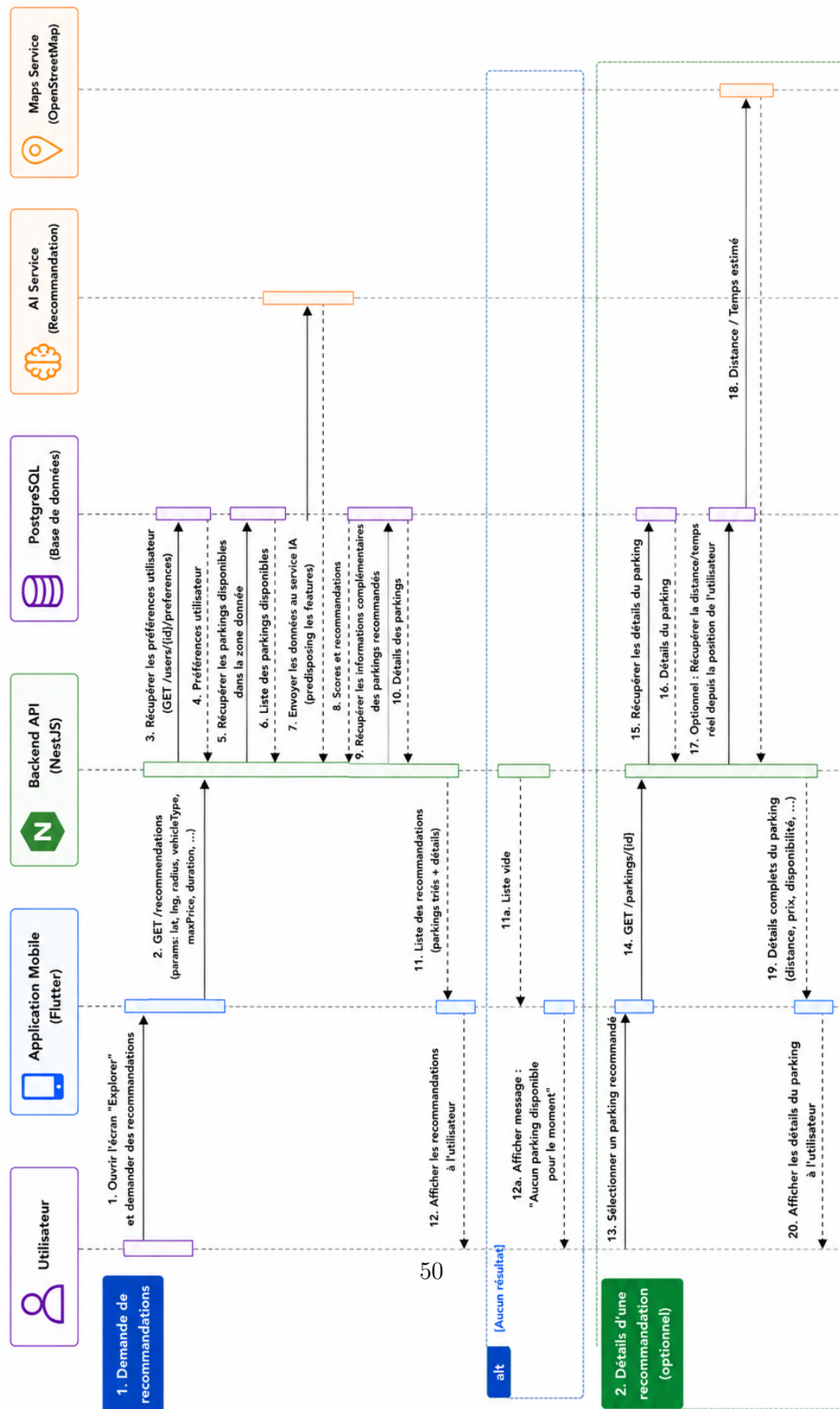


FIGURE 3.18 – Recommandation intelligente des parkings

3.5.6 Diagramme de séquence de l'ajout d'un parking privé

Ce diagramme présente le processus permettant à un propriétaire de proposer un parking privé sur la plateforme TuniPark. Après avoir renseigné les informations du parking et téléversé les images, l'application transmet la demande au Backend. Celui-ci vérifie les données, enregistre le parking avec le statut *PENDING* puis le rend disponible dans l'interface d'administration pour validation. Après examen, l'administrateur accepte ou rejette la demande. Le propriétaire est ensuite informé de la décision par une notification.

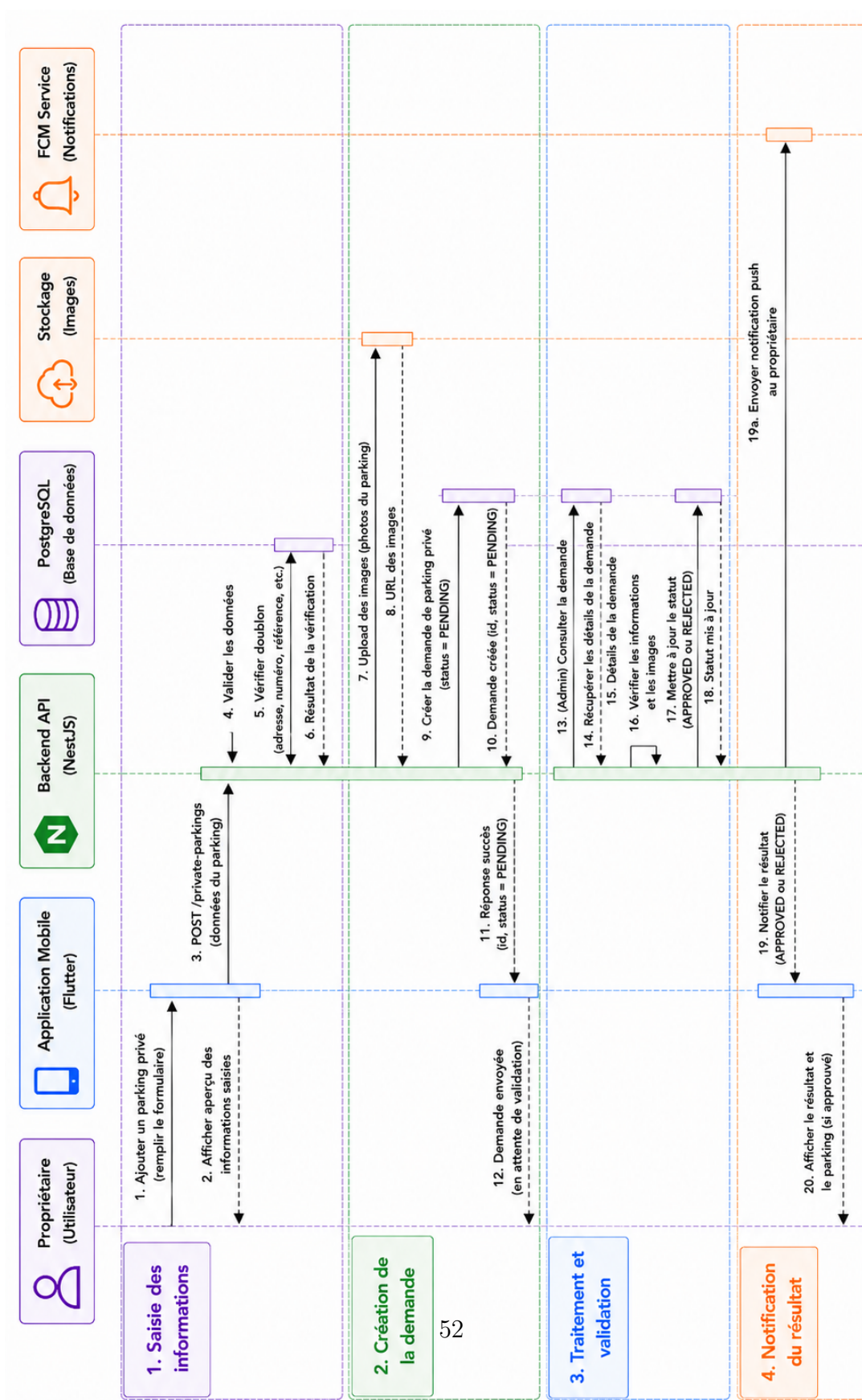


FIGURE 3.19 – Ajout d'un parking privé

3.5.7 Diagramme de séquence des notifications

Le diagramme de séquence des notifications présente le processus d'envoi d'une notification à un utilisateur de l'application **TuniPark**. Lorsqu'un événement important se produit, tel que la validation d'un parking privé, la confirmation d'un paiement ou l'activation d'une session de stationnement, le Backend crée une notification et le sauvegarde dans la base de données.

Il récupère ensuite les jetons FCM associés à l'utilisateur concerné et transmet le contenu de la notification à **Firestore Cloud Messaging** [9]. Ce service se charge de l'envoyer vers l'appareil mobile de l'utilisateur. La notification reste également disponible dans l'application afin de pouvoir être consultée ultérieurement et marquée comme lue.

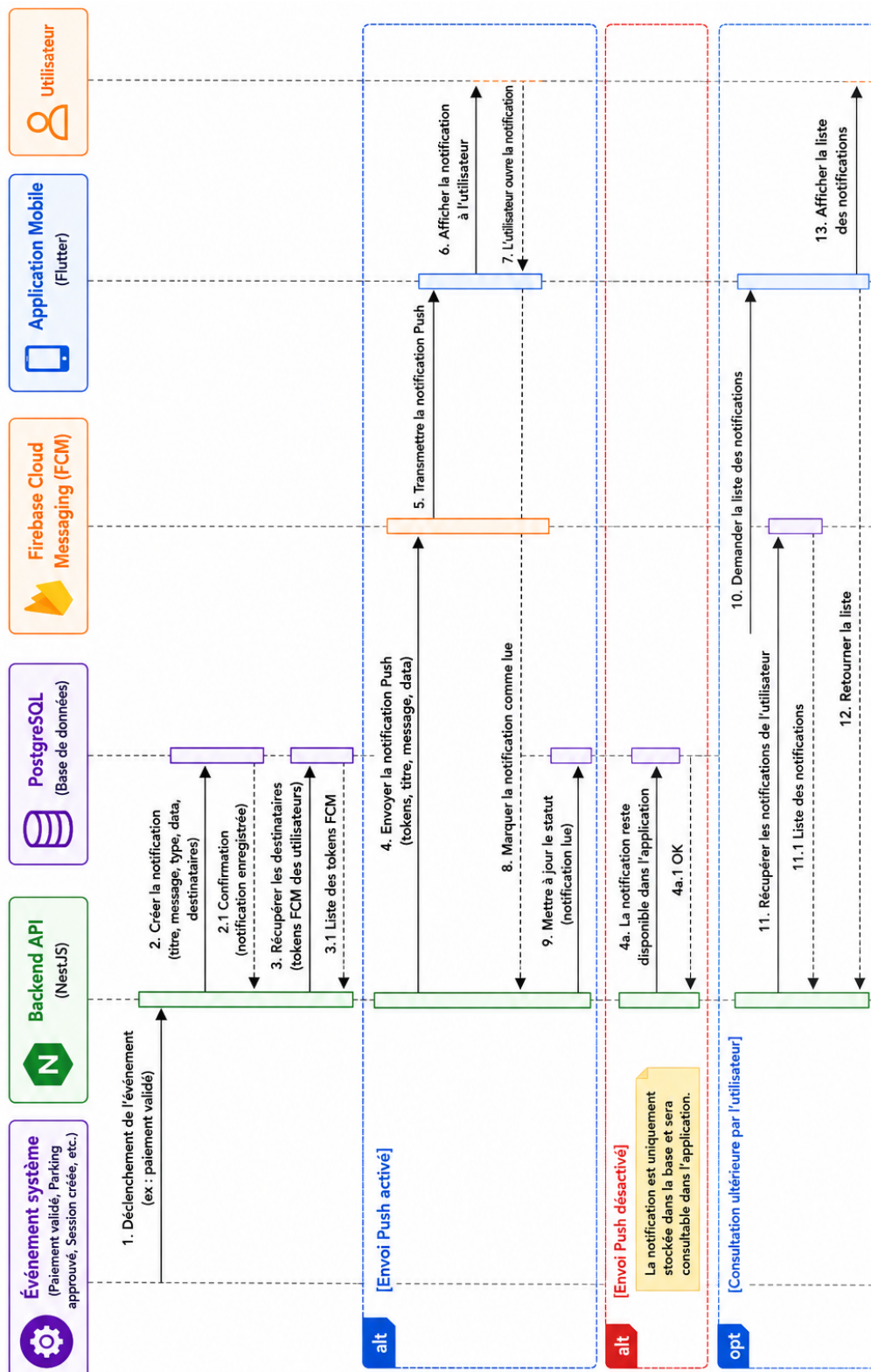


FIGURE 3.20 – Envoi des notifications

3.6 Conclusion

Ce chapitre a présenté la conception et l'architecture de l'application **TuniPark**. Il a décrit les architectures logique et physique du système, puis les principaux choix techniques retenus pour le développement des différents composants de la solution.

Le chapitre a également présenté la conception de la base de données ainsi que les principaux modèles UML utilisés pour représenter le fonctionnement de l'application. Le diagramme de classes a mis en évidence les principales entités du système et leurs relations, Alors que les diagrammes de séquence ont dépeint le déroulement des processus principaux, notamment l'authentification des utilisateurs, la création et le paiement d'une session de stationnement, la recommandation intelligente des parkings, l'ajout d'un parking privé ainsi que l'envoi des notifications.

L'ensemble de ces éléments de conception constitue une base robuste pour la phase d'implémentation de l'application. Le chapitre suivant abordera à l'implémentation de TuniPark et présentera les différents modules développés, les interfaces utilisateur ainsi que les principales fonctionnalités mises en œuvre.

Chapitre 4

Réalisation et Implémentation

4.1 Introduction

Après avoir présenté l'analyse des besoins ainsi que la conception de l'application TuniPark, ce chapitre décrit la phase de réalisation du projet. Il présente les principaux outils utilisés durant le développement ainsi que l'implémentation des différents modules constituant la solution.

Le chapitre présente tout d'abord l'environnement de développement adopté, puis décrit l'implémentation du Backend, de l'application mobile Flutter, de la plateforme web d'administration Angular, ainsi que l'intégration des différents services externes. Enfin, il présente le système de recommandation intelligent ainsi que les principales fonctionnalités développées.

4.2 Environnement de développement

La réalisation de l'application **TuniPark** a nécessité l'utilisation de plusieurs outils et technologies couvrant le développement mobile, le développement web, le développement Backend, l'IA, la gestion de la base de données, les tests, le déploiement tout comme l'intégration de différents services externes. Ces technologies ont été sélectionnées afin de garantir une architecture moderne, performante et facilement maintenable.

Outil / Technologie	Mise en œuvre dans le projet
Flutter	Réalisation de l'application mobile destinée aux utilisateurs.
Angular	Élaboration de la plateforme en ligne d'administration.
NestJS	Développement de l'API Backend et de la logique métier.
Prisma ORM	Couche d'accès aux données et gestion des modèles PostgreSQL.
PostgreSQL	Stockage des données de l'application (utilisateurs, parkings, sessions, paiements, notifications, etc.).
Python FastAPI	Développement du microservice d'IA utilisé pour la recommandation des parkings.
Scikit-learn (Random Forest)	Implémentation du modèle de recommandation intelligent.
Cloudinary	Stockage et gestion des images des parkings.
Firebase Cloud Messaging	Envoi des notifications Push vers les appareils mobiles.
Flouci	Gestion des paiements électroniques des sessions de stationnement.
Nodemailer	Envoi des courriers électroniques (vérification et récupération de compte).
Twilio	Gestion des codes OTP par SMS (prévu, non activé durant le projet).
Postman	Test et validation des services REST du Backend.
Render	Plateforme utilisée pour héberger le Backend NestJS, le microservice IA basé sur FastAPI, ainsi que la base de données PostgreSQL.
Vercel	Hébergement de la plateforme web d'administration Angular.
Git & GitHub	Gestion des versions et collaboration durant le développement.

TABLE 4.1 – Principaux outils et technologies utilisés lors de l'implémentation de Tunipark

4.3 Implémentation du Backend

Le Backend de **TuniPark** a été développé à l'aide du framework **NestJS**. Il constitue le cœur de l'application en assurant le traitement des requêtes provenant de l'application mobile Flutter et de la plateforme web d'administration Angular. Il applique les règles métier, communique avec la base de données PostgreSQL via Prisma ORM et interagit avec plusieurs services externes tels que Flouci, Firebase Cloud Messaging, Cloudinary, Nodemailer ainsi que le microservice d'IA.

Afin de garantir une architecture claire, évolutive et facilement maintenable, le Backend est organisé suivant une architecture modulaire. Chaque module est responsable d'un domaine fonctionnel spécifique, ce qui facilite le développement, les tests unitaires et l'évolution future de l'application.

4.3.1 Module d'authentification

Le module d'authentification assure la sécurisation de l'accès à l'application. Il prend en charge l'inscription, la connexion, le renouvellement des sessions ainsi que la récupération de mot de passe.

Suite à une authentification réussie, le Backend produit un jeton d'accès et un jeton de renouvellement basés sur JWT. Les accès aux différentes fonctionnalités sont ensuite contrôlés en fonction du rôle de l'utilisateur (*Utilisateur*, *Administrateur*, *Propriétaire de parking* ou *Auditeur*). Le module intègre également la vérification des adresses électroniques via Nodemailer ainsi que la vérification des numéros de téléphone par code OTP.

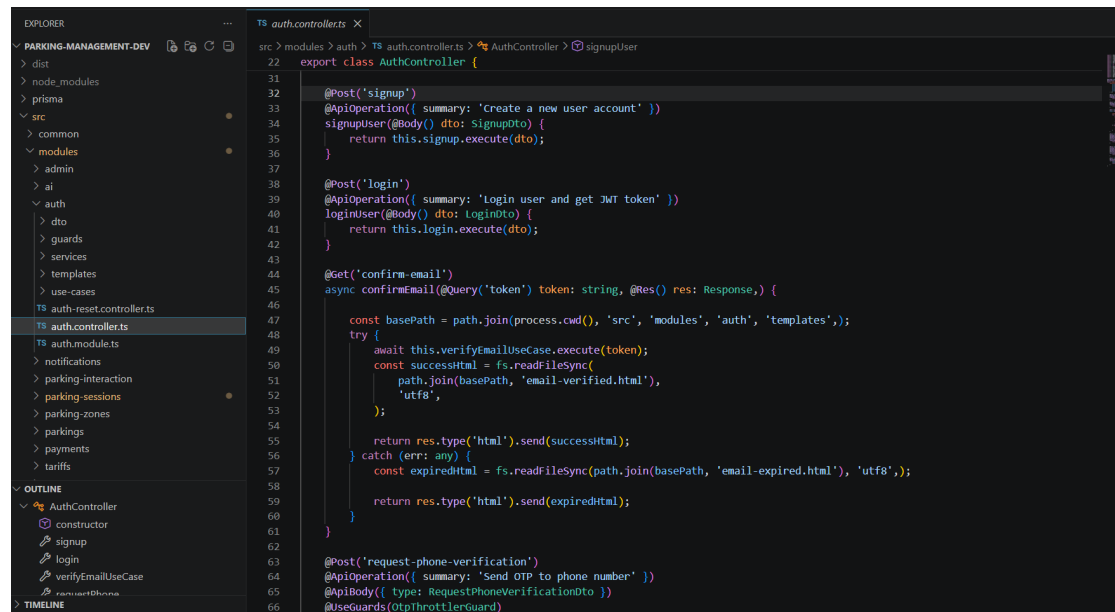


FIGURE 4.1 – Implémentation du module d'authentification

4.3.2 Module de gestion des parkings

Ce module permet la création, la modification et la gestion des parkings publics et privés. Les propriétaires peuvent publier un nouveau parking en renseignant ses caractéristiques, sa localisation, sa capacité, ses horaires, son tarif ainsi que les images correspondantes.

Les images sont automatiquement téléversées sur Cloudinary tandis que les informations descriptives sont enregistrées dans PostgreSQL. Les parkings privés sont initialement enregistrés avec le statut PENDING avant d'être validés ou rejetés par un administrateur.

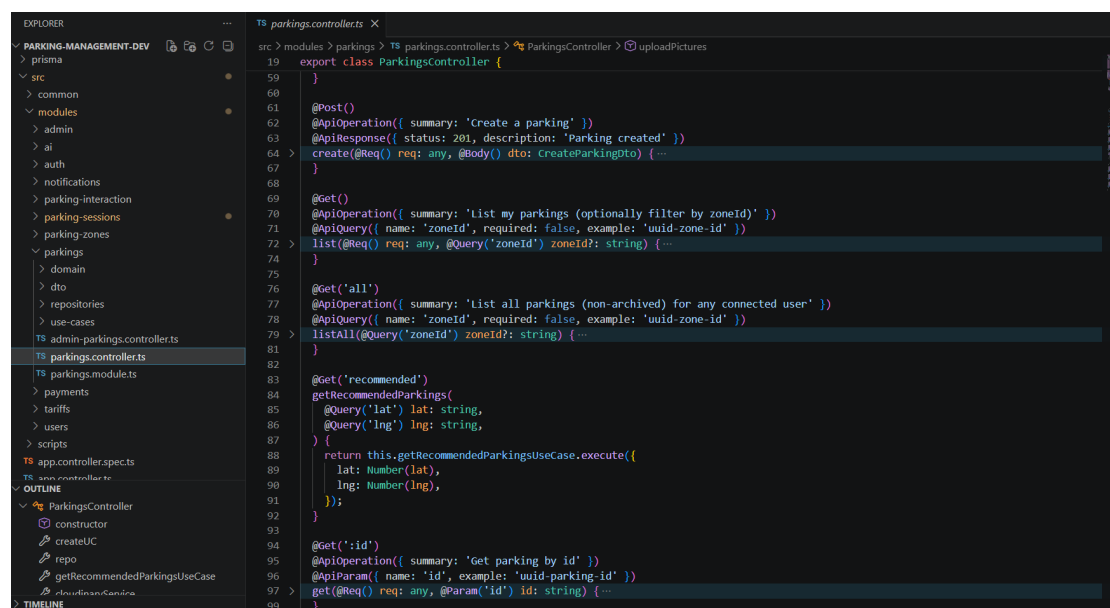


FIGURE 4.2 – Implémentation du module de gestion des parkings

4.3.3 Module de gestion des tarifs

Le module de gestion des tarifs permet d'associer une grille tarifaire à chaque parking. Il prend en charge les différents modes de facturation proposés par la plateforme, notamment le tarif à l'unité, le tarif journalier et le tarif mensuel.

Les administrateurs et les propriétaires de parkings peuvent définir ou modifier les tarifs en fonction des caractéristiques de chaque parking. Ces informations sont utilisées lors de la création des sessions de stationnement afin de calculer le montant à payer par l'utilisateur.

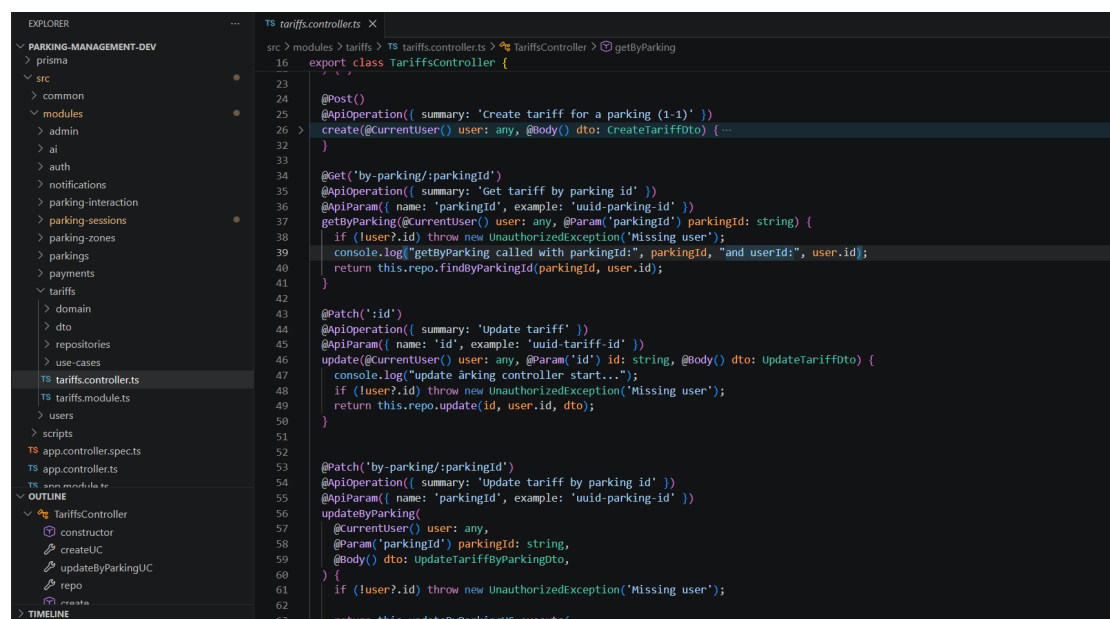


FIGURE 4.3 – Implémentation du module de gestion des tarifs

4.3.4 Module de gestion des sessions de stationnement

Le module des sessions permet aux utilisateurs de démarrer, prolonger et terminer une session de stationnement. Lorsqu'un utilisateur sélectionne un parking, le Backend vérifie sa disponibilité avant de créer une nouvelle session avec le statut **CREATED**.

Après validation du paiement, le statut de la session devient **ACTIVE**. Le module assure également le suivi des différentes étapes du stationnement jusqu'à la fermeture de la session, tout en conservant l'historique des opérations.

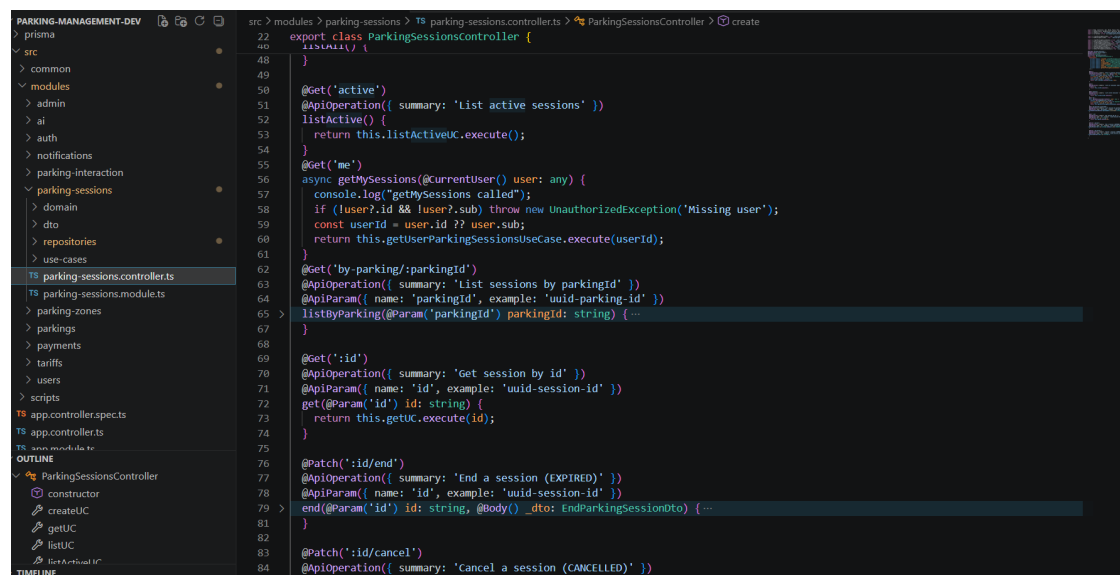


FIGURE 4.4 – Implémentation du module des sessions de stationnement

4.3.5 Module de paiement

Le module de paiement permet de gérer les transactions financières liées aux sessions de stationnement. Après la création d'une session, le Backend prépare automatiquement la transaction puis redirige l'utilisateur vers la plateforme de paiement **Flouci**.

Une fois le paiement confirmé, Flouci informe le Backend du résultat de la transaction. Celui-ci met alors à jour les informations de paiement ainsi que le statut de la session. Toutes les transactions sont enregistrées dans la base de données afin d'assurer leur traçabilité.

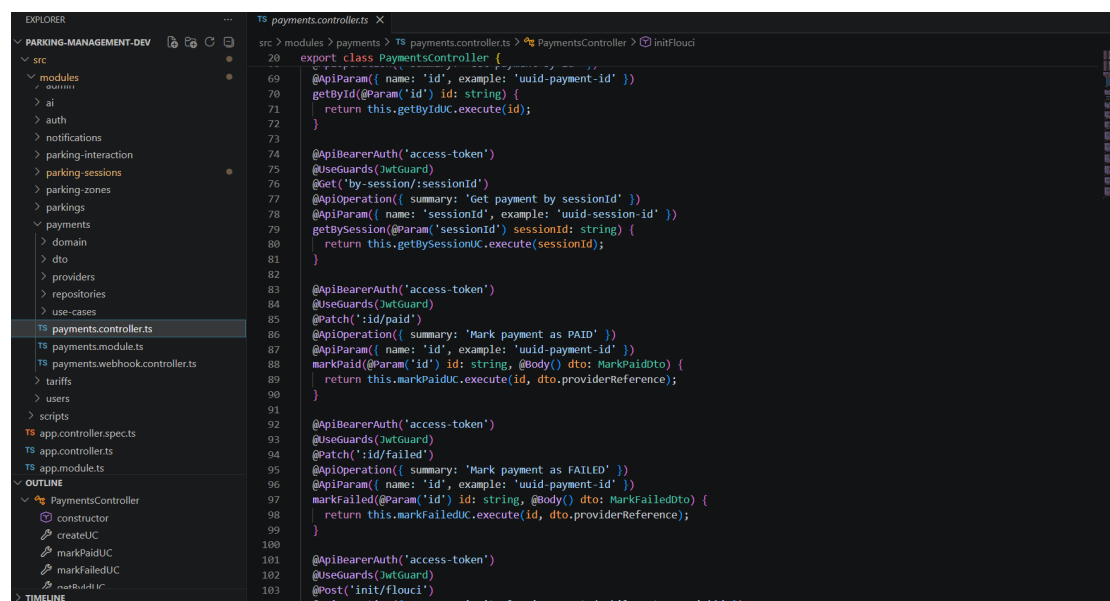


FIGURE 4.5 – Implémentation du module de paiement

4.3.6 Module de recommandation intelligente

Pour offrir une meilleure expérience aux utilisateurs, TuniPark intègre un système de recommandation basé sur l'IA. Le Backend collecte différentes informations relatives aux parkings, notamment leur distance, leur disponibilité, leur tarif ainsi que les statistiques d'utilisation.

Ces informations sont transmises à un microservice développé avec **FastAPI** utilisant un modèle **Random Forest**. Celui-ci calcule un score de recommandation pour chaque parking et retourne les résultats au Backend, qui les trie avant de les envoyer à l'application mobile. En cas d'indisponibilité du microservice, un algorithme de secours implémenté directement dans le Backend permet de garantir la continuité du service.

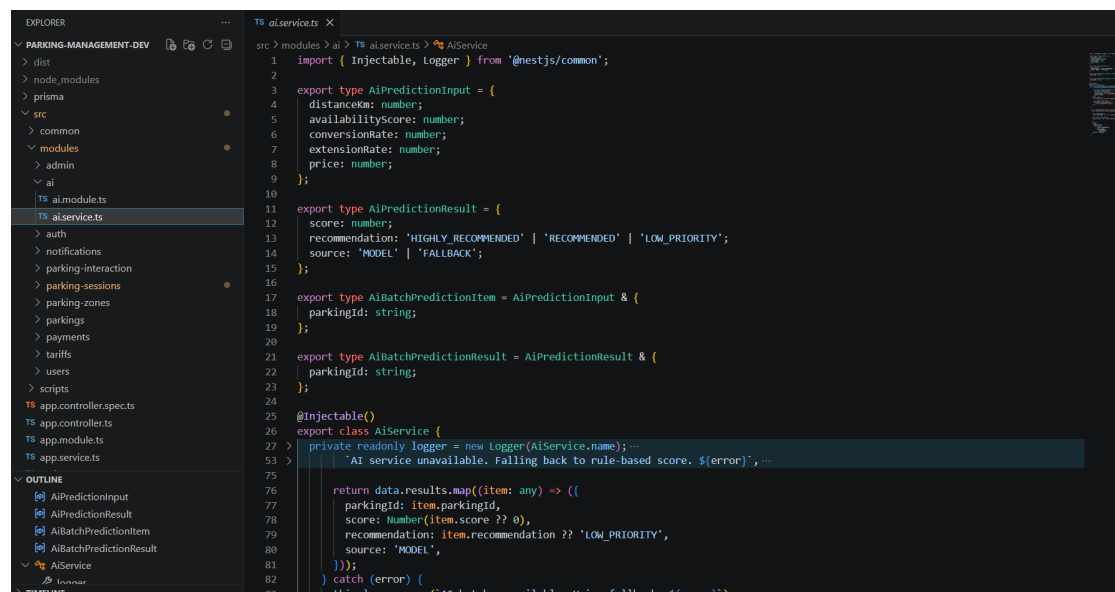


FIGURE 4.6 – Implémentation du module de recommandation intelligente

4.3.7 Module de notifications

Le module de notifications informe les utilisateurs des événements importants de l'application, tels que la validation d'un parking privé, la confirmation d'un paiement ou l'activation d'une session de stationnement.

Les notifications sont enregistrées dans la base de données et transmises aux appareils mobiles grâce à **Firestore Cloud Messaging**. Les jetons FCM des utilisateurs sont également gérés par ce module afin d'assurer la distribution des notifications Push.

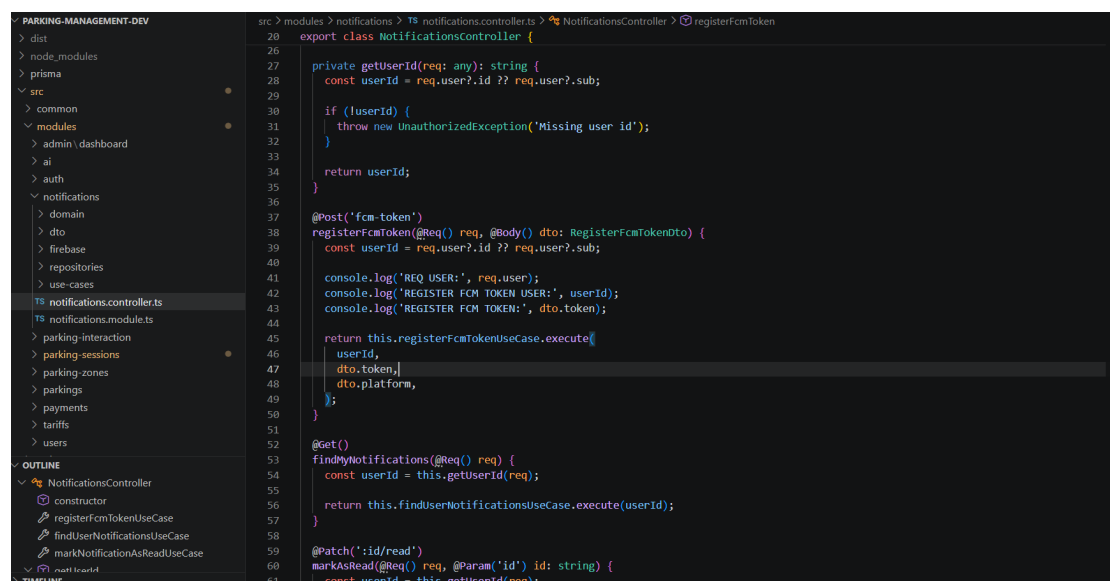


FIGURE 4.7 – Implémentation du module de notifications

4.3.8 Module des interactions avec les parkings

Le module des interactions enregistre les différentes actions réalisées par les utilisateurs sur les parkings. Chaque consultation d'un parking, démarrage de session ou prolongation de stationnement est enregistrée dans la base de données.

Ces informations permettent de produire des statistiques d'utilisation et sont exploitées par le système de recommandation intelligent afin d'évaluer la popularité des parkings et d'améliorer la pertinence des recommandations proposées aux utilisateurs.

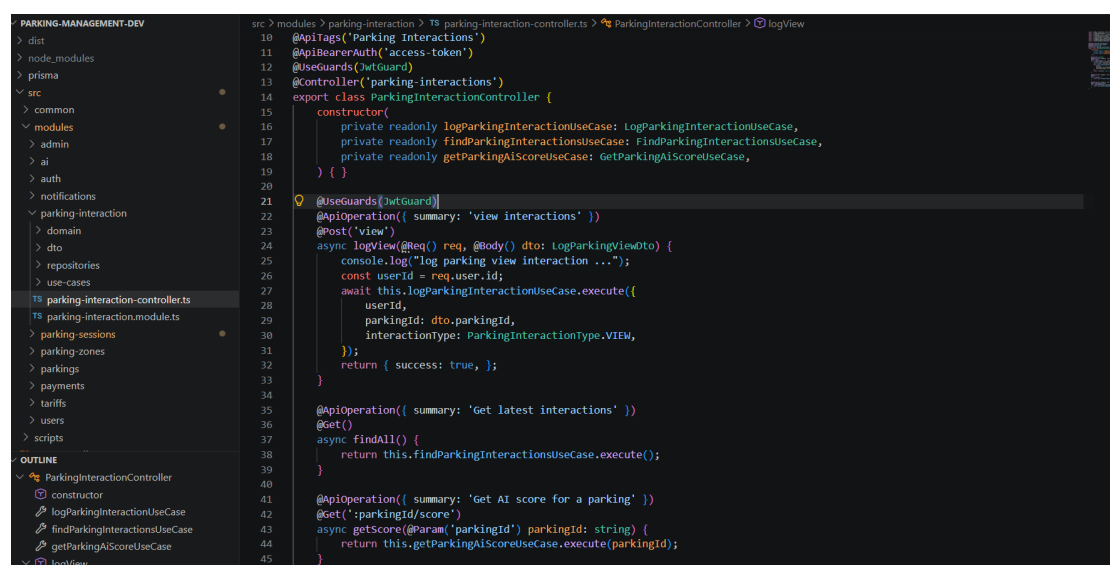


FIGURE 4.8 – Implémentation du module des interactions avec les parkings

4.3.9 Module de gestion des utilisateurs

Ce module permet de gérer les comptes des utilisateurs de la plateforme. Il prend en charge la création des comptes, la mise à jour des informations personnelles, la gestion des rôles ainsi que la vérification des adresses électroniques et des numéros de téléphone.

Le module assure également l'administration des comptes, notamment le blocage d'un utilisateur, la récupération du mot de passe et la gestion des différents profils tels que les utilisateurs, les propriétaires de parkings, les auditeurs et les administrateurs.



FIGURE 4.9 – Implémentation du module de gestion des utilisateurs

4.4 Implémentation de l'application mobile

TuniPark a été développée à l'aide du framework **Flutter**. Elle permet aux utilisateurs de rechercher des parkings, de gérer leurs sessions de stationnement, d'effectuer des paiements sécurisés et de consulter leur historique. L'application adopte une architecture modulaire reposant sur le principe du *Clean Architecture*, ce qui facilite la maintenance, les évolutions futures et la réutilisation du code.

Les interfaces ont été conçues afin d'offrir une navigation simple et intuitive tout en exploitant les fonctionnalités natives du smartphone, notamment la géolocalisation, les notifications Push et le stockage sécurisé des informations d'authentification.

4.4.1 Authentification

Ces interfaces permettent aux utilisateurs de créer un compte, de se connecter et de récupérer leur mot de passe. La plateforme prend en charge l'authentification par adresse électronique ou numéro de téléphone. En cas d'oubli du mot de passe, un code OTP est envoyé afin de vérifier l'identité de l'utilisateur avant de lui permettre de définir un nouveau mot de passe.

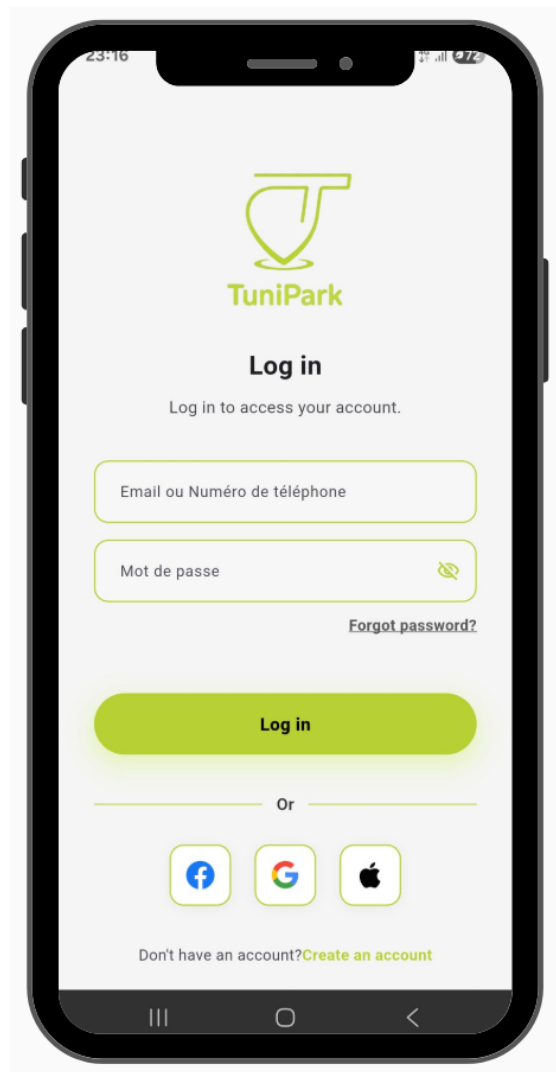


FIGURE 4.10 – Interfaces d’authentification

4.4.2 Exploration des parkings

Après authentification, l'utilisateur accède à la page d'exploration. Celle-ci affiche les parkings disponibles à proximité sous forme de carte interactive accompagnée de recommandations générées par le système intelligent. L'utilisateur a la possibilité de visualiser les détails de chaque parking avant de démarrer une session.



FIGURE 4.11 – Exploration des parkings

4.4.3 Création d'une session de stationnement

Pour démarrer une session de stationnement, l'utilisateur sélectionne un parking puis renseigne les informations nécessaires concernant son véhicule et la durée souhaitée. Une fois les informations validées, l'application lance automatiquement le processus de paiement.

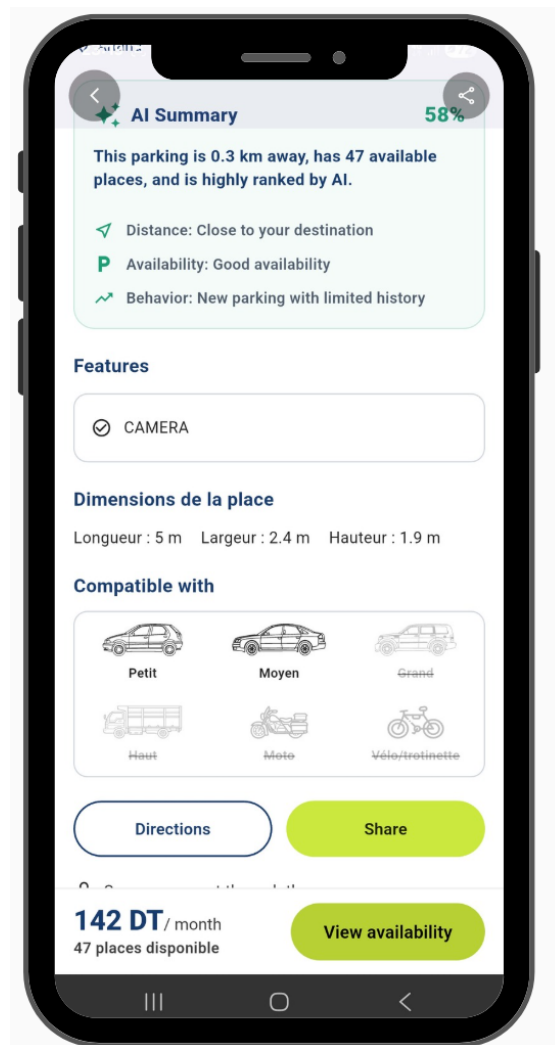


FIGURE 4.12 – Création d'une session de stationnement

4.4.4 Paiement d'une session

Les paiements sont réalisés via la plateforme Flouci. Après validation de la transaction, l'application reçoit la confirmation du Backend et active automatiquement la session de stationnement.

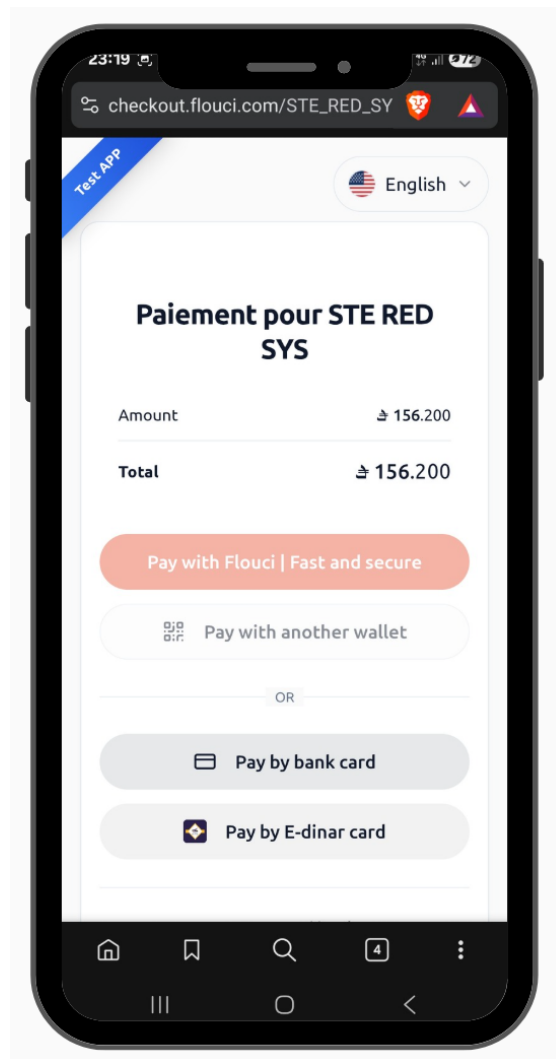


FIGURE 4.13 – Paiement d’une session

4.4.5 Suivi de la session

Une fois le paiement confirmé, l’usager a la possibilité de visualiser les détails concernant sa session actuelle, notamment le parking sélectionné, la durée restante et les différentes actions disponibles telles que la prolongation ou l’arrêt de la session.

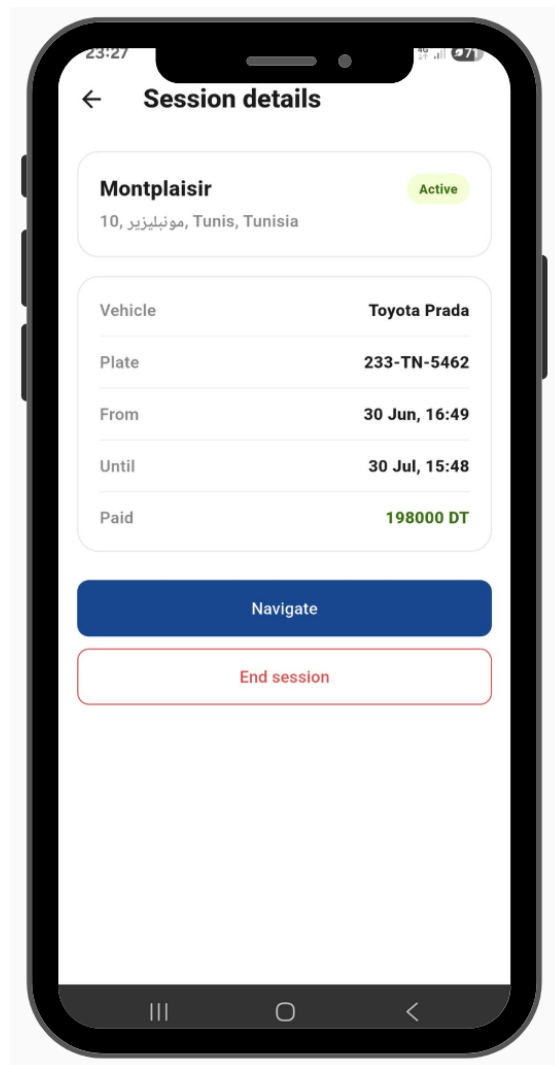


FIGURE 4.14 – Suivi d’une session active

4.4.6 Ajout d’un parking privé

Les propriétaires peuvent proposer un parking privé en renseignant les informations générales, les caractéristiques, les tarifs et les photographies du parking. Après soumission, la demande est transmise à l’administrateur pour validation.

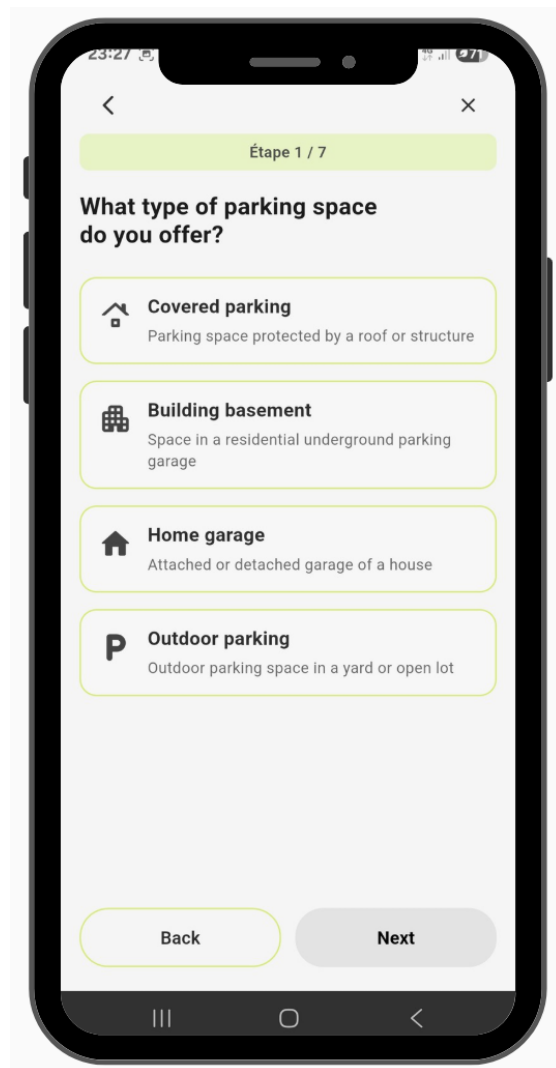


FIGURE 4.15 – Ajout d'un parking privé

4.4.7 Profil utilisateur

L'application permet aux utilisateurs de consulter et de modifier leurs informations personnelles, de gérer leurs véhicules enregistrés ainsi que d'accéder à leur historique de stationnement.

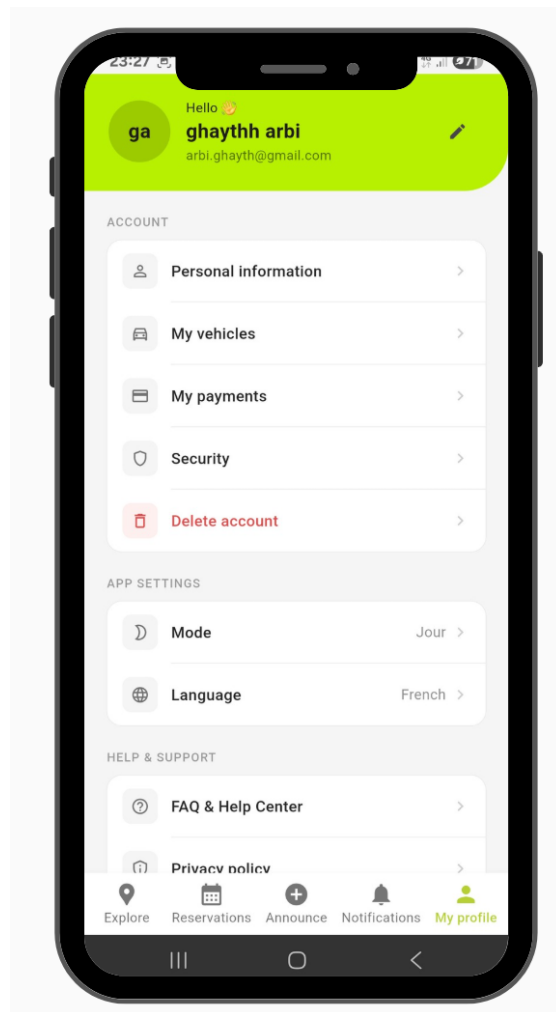


FIGURE 4.16 – Interface du profil utilisateur

4.4.8 Notifications

Les notifications permettent d'informer l'utilisateur des événements importants de l'application, notamment la validation d'un parking privé, la confirmation d'un paiement, le début d'une session ou encore les rappels liés au stationnement.

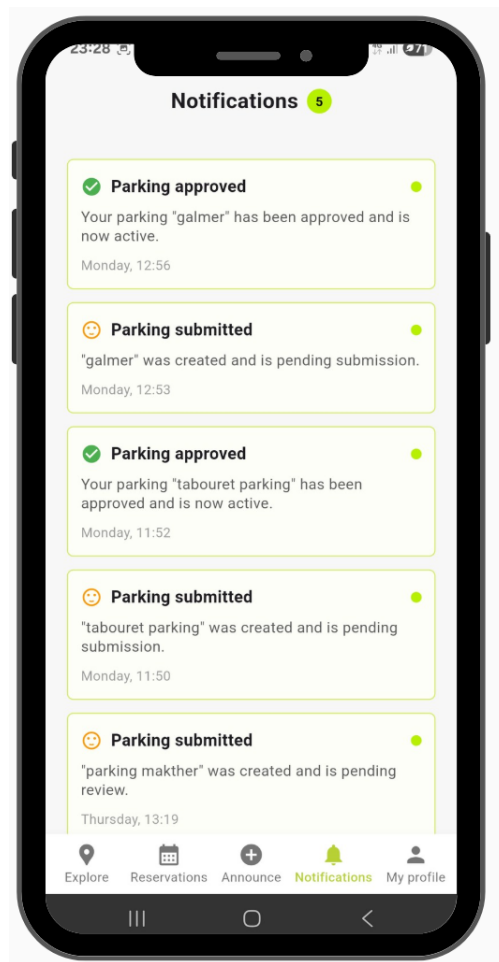


FIGURE 4.17 – Interface des notifications

4.5 Fonctionnalités administratives

La plateforme web d'administration a été développée avec **Angular**. Elle est exclusivement accessible aux utilisateurs disposant des privilèges administratifs. Après authentification, l'administrateur accède à un tableau de bord lui permettant de superviser les principales opérations de l'application TuniPark.

L'interface d'administration centralise la gestion des utilisateurs, des parkings publics et privés, des zones de stationnement, des sessions, des paiements ainsi que des notifications. Elle permet également de valider ou refuser les requêtes d'ajout de parkings privés, de consulter les statistiques d'utilisation de la plateforme et de gérer les informations nécessaires au bon fonctionnement du système.

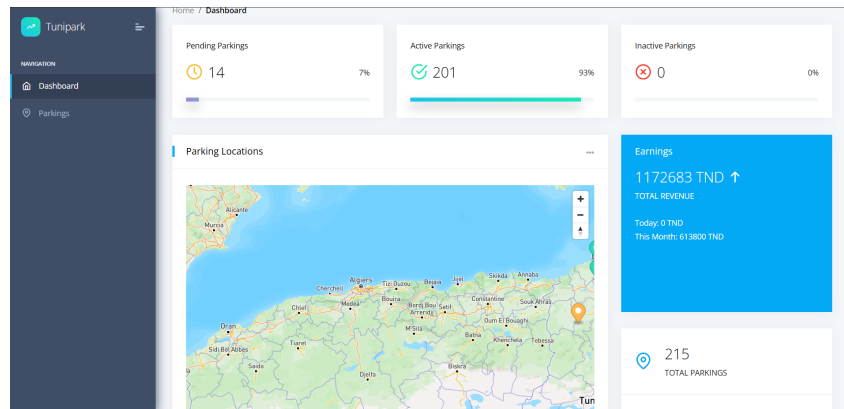


FIGURE 4.18 – Tableau de bord de l'administration

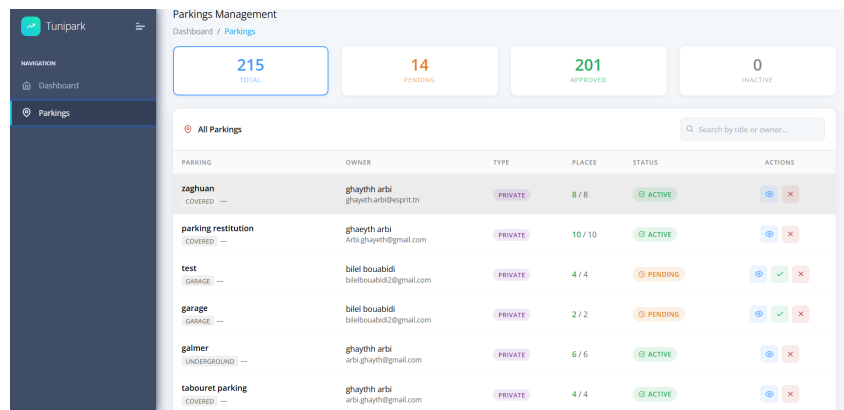


FIGURE 4.19 – Interface de gestion des ressources

4.6 Présentation du déploiement

Après le développement et la validation des différentes fonctionnalités, l'application **TuniPark** a été déployée dans un contexte de production afin de permettre son utilisation dans des conditions réelles. Les différents composants de la plateforme sont hébergés sur des services cloud adaptés à leur rôle, garantissant leur disponibilité, leur maintenance et leur évolutivité.

Le Backend développé avec **NestJS** est hébergé sur la plateforme **Render**. Cette plateforme assure l'exécution continue de l'API REST ainsi que la communication avec la base de données **PostgreSQL**, également hébergée sur **Render**. Cette configuration permet au Backend de traiter les requêtes provenant de l'application mobile et de la plateforme web d'administration de manière sécurisée.

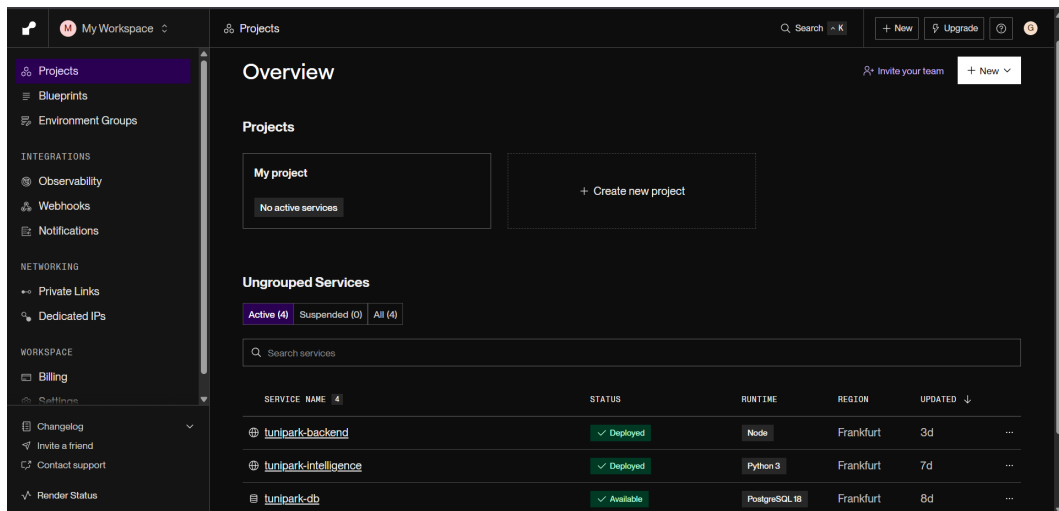


FIGURE 4.20 – Déploiement du Backend et de la base de données sur Render

La plateforme web d'administration développée avec **Angular** est déployée sur **Vercel**. Cette solution facilite le déploiement continu de l'application et permet la publication rapide des nouvelles versions de l'interface d'administration.

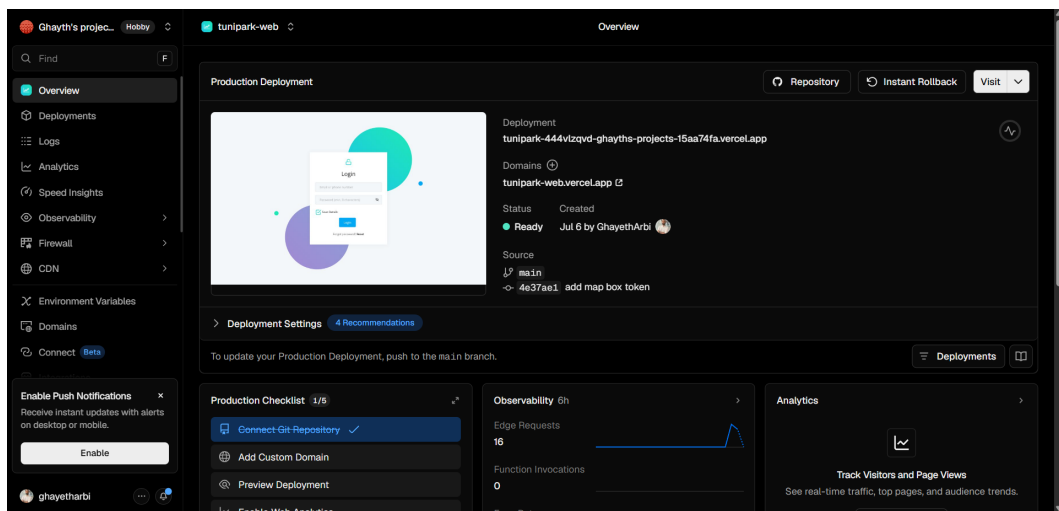


FIGURE 4.21 – Déploiement de la plateforme web d'administration sur Vercel

En complément de ces services, plusieurs solutions externes ont été intégrées pour garantir le fonctionnement optimal de l'application. Les images des parkings sont stockées sur **Cloudinary**, les notifications Push sont envoyées grâce à **Firestore Cloud Messaging**, les paiements électroniques sont traités par **Flouci** et les courriers électroniques sont transmis à l'aide de **Nodemailer**. L'envoi des codes

OTP par SMS a été prévu via **Twilio**, mais cette fonctionnalité n'a pas été activée durant la période du projet en raison de l'absence d'un abonnement au service.

Cette infrastructure de déploiement permet aux différents composants de fonctionner de manière indépendante tout en garantissant une communication sécurisée entre l'application mobile, la plateforme web d'administration, le Backend, la base de données ainsi que les différents services externes intégrés à la solution.

4.7 Conclusion

Ce chapitre a présenté la réalisation de l'application **TuniPark** en décrivant l'environnement de développement, les principaux modules du Backend, les interfaces de l'application mobile ainsi que la plateforme web d'administration.

Il a également présenté les différentes fonctionnalités implémentées, notamment l'authentification sécurisée, la gestion des parkings, les sessions de stationnement, les paiements électroniques, les notifications Push et le système de recommandation intelligent basé sur un modèle Random Forest.

Enfin, le chapitre a décrit l'architecture de déploiement de la solution en mettant en évidence l'intégration des différents services cloud utilisés durant le projet. Le prochain chapitre portera sur tests et sur la validation de l'application afin d'évaluer son bon fonctionnement et de vérifier que les exigences définies au cours de l'analyse ont été satisfaites.

Chapitre 5

Tests, Validation et Résultats

5.1 Introduction

Après l'implémentation des principaux modules de l'application **TuniPark**, une phase de tests et de validation a été réalisée afin de vérifier le bon fonctionnement des fonctionnalités développées. Cette étape avait pour objectif de s'assurer que l'application mobile, la plateforme web d'administration, les services Backend ainsi que les différents services externes fonctionnaient correctement de manière intégrée.

Ce chapitre présente la stratégie de test adoptée durant le développement, les outils utilisés pour la validation des API et des fonctionnalités, les principaux scénarios testés ainsi que les résultats obtenus. Il met également en évidence les difficultés rencontrées au cours de la phase de validation.

5.2 Stratégie de test

Les tests ont été réalisés progressivement tout au long du développement de l'application. Chaque fonctionnalité a été validée individuellement avant d'être testée après son intégration avec les autres modules. Cette approche a permis d'identifier rapidement les anomalies et de garantir le bon fonctionnement global de la plateforme.

La validation a principalement porté sur les fonctionnalités essentielles de TuniPark, notamment l'authentification des utilisateurs, la gestion des parkings, la création des sessions de stationnement, les paiements électroniques, les notifications Push ainsi que le système de recommandation intelligent.

Les principaux objectifs de cette phase de validation étaient les suivants :

- vérifier le bon fonctionnement de l'authentification et du contrôle d'accès ;
- valider les opérations de gestion des parkings ;

-
- confirmer le bon déroulement de la création et du paiement des sessions de stationnement ;
 - vérifier l’envoi des notifications Push ;
 - valider le fonctionnement du système de recommandation intelligent ;
 - s’assurer de la bonne communication entre les différents services de la plateforme.

5.3 Outil de test des API

Durant la phase de validation du Backend, l’interface **Swagger** intégrée à **NestJS** a été utilisée afin de tester directement les API développées. Cet outil a permis d’envoyer des requêtes HTTP, d’analyser les réponses du serveur et de vérifier le bon fonctionnement des différents points d’accès sans passer systématiquement par l’application mobile.

Dans le cadre de ce projet, Swagger [18] a été utilisé pour tester les principales fonctionnalités de l’application, notamment l’authentification, la gestion des utilisateurs, des parkings, des sessions de stationnement, des paiements, des notifications ainsi que les routes protégées nécessitant un jeton JWT. Il a également permis de vérifier les paramètres des requêtes, les réponses retournées par le serveur ainsi que les différents codes de statut HTTP.

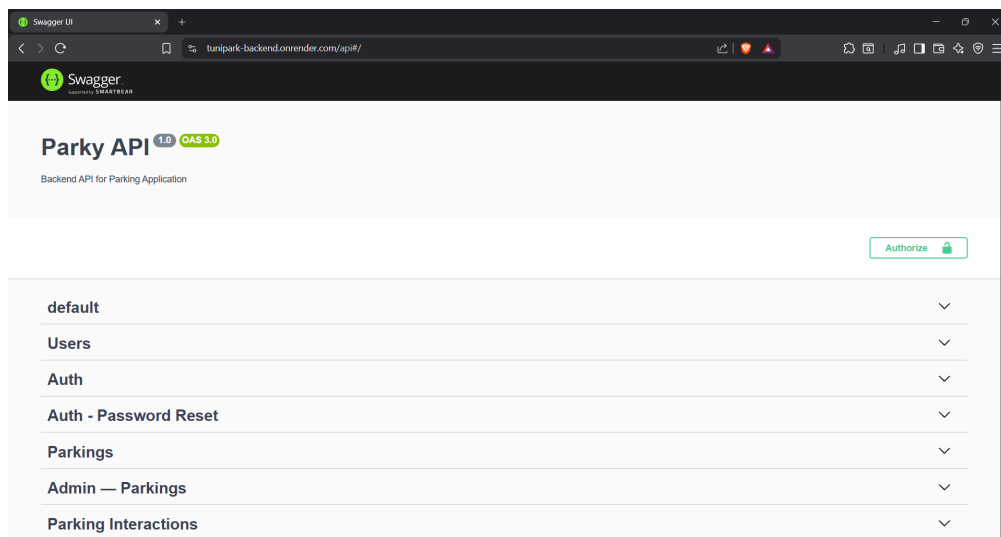


FIGURE 5.1 – Documentation et tests des API à l’aide de Swagger

5.4 Validation des API du Backend

La validation des API a débuté par le test du point d'accès d'authentification. Après une connexion réussie, le Backend génère un *Access Token* ainsi qu'un *Refresh Token*. Ces jetons sont ensuite utilisés pour accéder aux différentes routes sécurisées de l'application et vérifier le contrôle d'accès selon les rôles des utilisateurs.

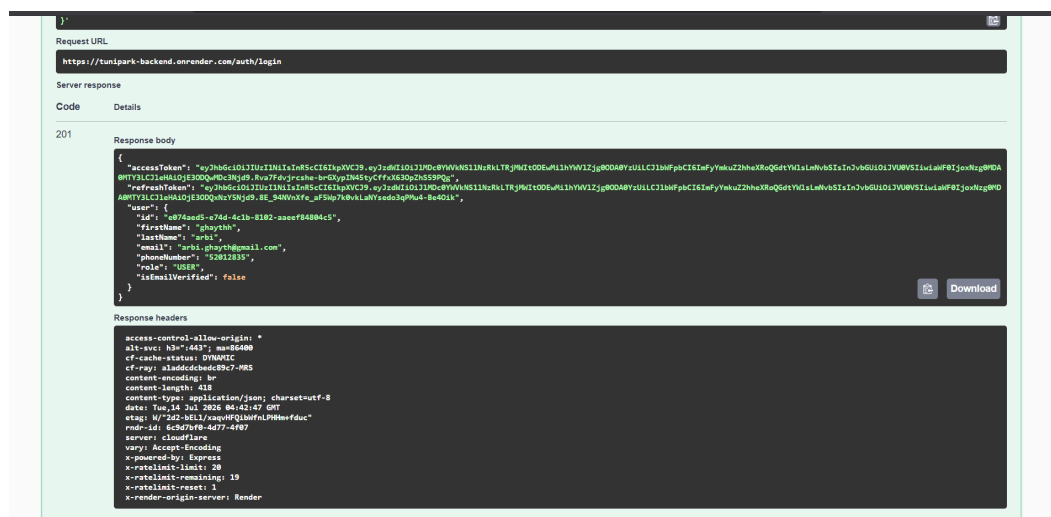


FIGURE 5.2 – Validation de l'API d'authentification avec Swagger

Des tests complémentaires ont ensuite été réalisés sur les principales API de l'application, notamment la gestion des parkings, la création et le paiement des sessions de stationnement, les notifications, le système de recommandation ainsi que les opérations d'administration. Ces essais ont confirmé que les requêtes valides étaient correctement traitées et que le Backend retournait les réponses ainsi que les codes d'erreur appropriés lorsque les données fournies étaient incomplètes ou invalides.

5.5 Tests fonctionnels

Les tests fonctionnels ont été réalisés afin de vérifier le bon fonctionnement de l'application **TuniPark** du point de vue des utilisateurs et des administrateurs. Chaque fonctionnalité a été testée dans des conditions d'utilisation réelles afin de s'assurer que les résultats obtenus correspondaient au comportement attendu.

Les essais ont porté sur les principales fonctionnalités de l'application, notamment l'authentification, la gestion des parkings, la création des sessions de

stationnement, les paiements électroniques, les notifications Push ainsi que le système de recommandation intelligent. Les résultats obtenus ont permis de confirmer le bon fonctionnement de l'ensemble des modules développés.

5.5.1 Tests d'authentification et de contrôle d'accès

Les tests d'authentification ont permis de vérifier la connexion des utilisateurs, la génération des jetons JWT ainsi que le contrôle d'accès aux ressources protégées. Le mécanisme de renouvellement automatique des jetons à l'aide du *Refresh Token* a également été validé.

Test	Comportement attendu	Résultat
Connexion avec des identifiants valides	L'utilisateur accède à l'application.	Réussi
Connexion avec des identifiants invalides	Message d'erreur affiché.	Réussi
Accès sans jeton JWT	La requête est refusée.	Réussi
Renouvellement de l'Access Token	Un nouveau jeton est généré via le Refresh Token.	Réussi

TABLE 5.1 – Tests d'authentification et de contrôle d'accès

5.5.2 Tests de gestion des parkings

Les fonctionnalités de gestion des parkings ont été testées aussi bien du côté propriétaire que du côté administrateur. Les essais ont permis de vérifier la création d'un parking privé, sa validation par un administrateur ainsi que la mise à jour de ses informations.

Test	Comportement attendu	Résultat
Ajouter un parking privé	Parking créé avec le statut PENDING.	Réussi
Valider un parking	Le statut devient ACTIVE.	Réussi
Rejeter un parking	Le parking reste indisponible.	Réussi
Modifier les informations d'un parking	Les modifications sont enregistrées.	Réussi

TABLE 5.2 – Tests de gestion des parkings

5.5.3 Tests des sessions de stationnement et des paiements

Les scénarios de création d'une session de stationnement et de paiement via Flouci ont été validés afin de vérifier le bon déroulement du processus complet.

Test	Comportement attendu	Résultat
Créer une session	Une session est créée avec le statut CREATED .	Réussi
Paielement Flouci validé	La session passe au statut ACTIVE .	Réussi
Paielement annulé	La session n'est pas activée.	Réussi
Fin de session	Le paielement est enregistré correctement.	Réussi

TABLE 5.3 – Tests des sessions de stationnement et des paiements

5.5.4 Tests des notifications

Les notifications internes ainsi que les notifications Push envoyées via Firebase Cloud Messaging ont été testées afin de vérifier leur création, leur envoi et leur affichage dans l'application.

Test	Comportement attendu	Résultat
Créer une notification	Notification enregistrée en base de données.	Réussi
Notification Push FCM	Réception sur le téléphone mobile.	Réussi
Marquer une notification comme lue	Mise à jour du statut de lecture.	Réussi

TABLE 5.4 – Tests des notifications

5.5.5 Tests du système de recommandation

Le système de recommandation intelligent a été testé afin de vérifier le fonctionnement du modèle Random Forest et la génération des recommandations de parkings.

Test	Comportement attendu	Résultat
Demande de recommandations	Liste des parkings recommandés affichée.	Réussi
Microservice IA disponible	Calcul des scores avec Random Forest.	Réussi
Indisponibilité du microservice	Utilisation de l'algorithme de secours.	Réussi

TABLE 5.5 – Tests du système de recommandation

5.6 Validation de la base de données

La validation de la base de données **PostgreSQL** a été réalisée en vérifiant que les différentes opérations effectuées depuis l'application entraînaient bien les modifications attendues au niveau des données. Les résultats obtenus ont été contrôlés à travers les réponses des API ainsi que les journaux d'exécution du Backend.

Les contrôles ont principalement concerné la mise en place des comptes utilisateurs, l'intégration et l'approbation des parkings, l'établissement des sessions de stationnement, le suivi des paiements, les alertes, ainsi que les communications employées par le système de recommandation. Les différents tests ont confirmé que les opérations de création, de modification et de suppression étaient correctement prises en compte et que la cohérence entre les entités associées était bien préservée.

id	createdAt	email	emailResetExpiresAt	emailResetOTPHash	firstName
45823dde-e372-48b3-ba94-d668243f00c8	2026-06-16T14:06:58.668Z	owner1@tunipark.tn	NULL	NULL	Owner1
641cc156-b527-43d9-8d7a-de48007e55a7	2026-06-16T14:06:58.668Z	owner2@tunipark.tn	NULL	NULL	Owner2
0878f4e4-7969-419b-b3c9-d034349a972d	2026-06-16T14:06:58.668Z	owner3@tunipark.tn	NULL	NULL	Owner3
e074aed5-e74d-4c1b-8102-aaef84804c5	2026-02-24T08:44:55.683Z	arbi.ghayth@gmail.com	NULL	NULL	ghayth
2d67163e-1ce0-4dce-aa17-9e12bcae8494	2026-06-16T14:06:58.668Z	owner4@tunipark.tn	NULL	NULL	Owner4
990496bd-9434-4cf6-9d74-3381dea9b65a	2026-06-16T14:06:58.668Z	owner5@tunipark.tn	NULL	NULL	Owner5
b68bba97-144d-4ce2-adc6-bbe97f16b21	2026-06-16T14:06:58.668Z	owner6@tunipark.tn	NULL	NULL	Owner6
8d540549-761b-4093-be09-676675cfaf0	2026-06-16T14:06:58.668Z	owner7@tunipark.tn	NULL	NULL	Owner7
2eeb83be-473a-42aa-8ab0-8467c366858d	2026-06-16T14:06:58.668Z	owner8@tunipark.tn	NULL	NULL	Owner8
7fc3655a-8ace-4f3a-866d-690fc18df566	2026-06-16T14:06:58.668Z	owner9@tunipark.tn	NULL	NULL	Owner9
a1c34790-b361-491e-abe4-6e9c5fc2bd37	2026-06-16T14:06:58.668Z	owner10@tunipark.tn	NULL	NULL	Owner10
ce3a0744-c2b7-417b-863e-1256cae1884b	2026-06-16T14:06:58.668Z	driver1@tunipark.tn	NULL	NULL	Driver1
c00578a3-df62-4a65-889a-43e78328d39	2026-06-16T14:06:58.668Z	driver2@tunipark.tn	NULL	NULL	Driver2
e76be9c7-3752-47c1-8719-cc6ba3f83afd	2026-06-16T14:06:58.668Z	driver3@tunipark.tn	NULL	NULL	Driver3
4978f978-7712-471e-aab6-dba711af280d	2026-06-16T14:06:58.668Z	driver4@tunipark.tn	NULL	NULL	Driver4

FIGURE 5.3 – Validation des données enregistrées dans PostgreSQL

5.7 Validation des services externes

Plusieurs fonctionnalités de l'application **TuniPark** reposent sur des services externes. Des tests ont donc été réalisés afin de vérifier leur bonne intégration avec le Backend. Les validations ont principalement concerné le paiement électronique avec **Flouci**, l'envoi des notifications Push via **Firebase Cloud Messaging** ainsi que le stockage des images sur **Cloudinary**.

5.7.1 Validation des paiements avec Flouci

L'intégration de Flouci a été validée lors de la création d'une session de stationnement. Après la sélection d'un parking, l'utilisateur a été redirigé vers la plateforme de paiement. Une fois la transaction confirmée, Flouci a transmis le résultat au Backend, qui a mis à jour le statut du paiement ainsi que celui de la session de stationnement.



FIGURE 5.4 – Validation d'un paiement avec Flouci

Les essais réalisés ont confirmé le bon fonctionnement du processus de paiement et la mise à jour correcte des informations enregistrées dans la base de données.

5.7.2 Validation des notifications Firebase

Les notifications Push ont été testées à l'aide de Firebase Cloud Messaging. Lorsqu'un événement nécessitant une notification se produit, le Backend récupère le jeton FCM de l'utilisateur et transmet la notification au service Firebase, qui l'envoie ensuite vers l'appareil mobile.

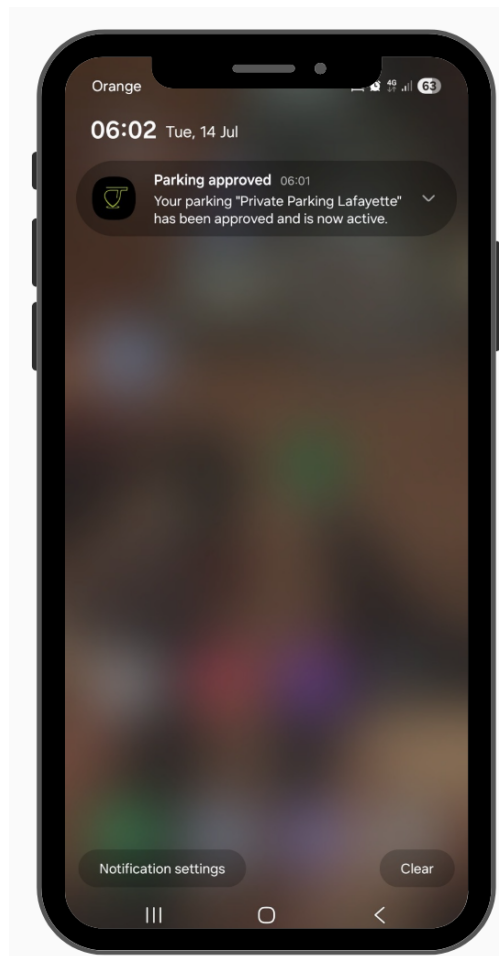


FIGURE 5.5 – Validation des notifications Push avec Firebase Cloud Messaging

Les notifications ont été correctement reçues sur les appareils de test et enregistrées dans l'application.

5.7.3 Validation du stockage Cloudinary

Le service Cloudinary a été utilisé pour stocker les images des parkings ajoutées par les propriétaires. Les tests ont consisté à téléverser plusieurs images depuis l'application puis à vérifier leur enregistrement sur Cloudinary ainsi que leur affichage après récupération par le Backend.

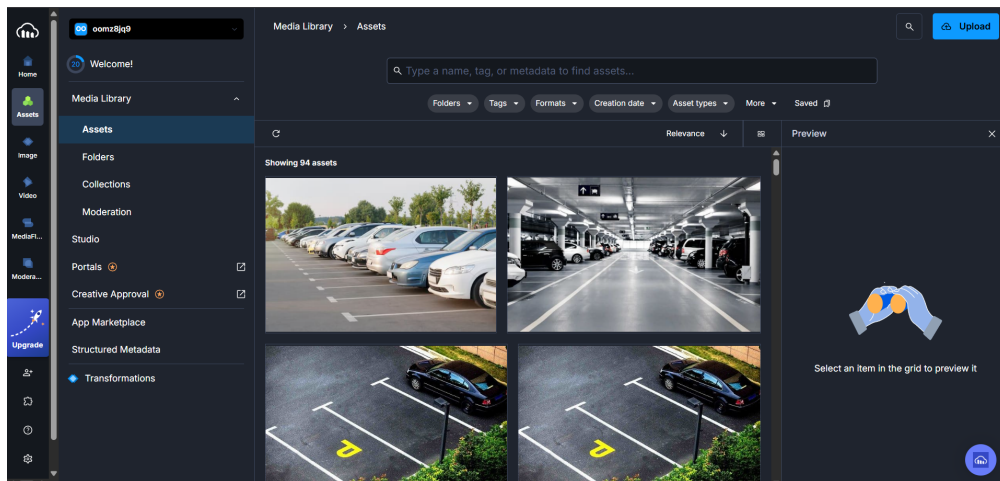


FIGURE 5.6 – Validation du stockage des images avec Cloudinary

Les essais ont confirmé que les images étaient correctement téléversées, stockées puis affichées dans l'application sans perte de qualité.

5.8 Résultats et discussion

Les différentes phases de validation ont confirmé le bon fonctionnement des principales fonctionnalités de l'application **TuniPark**. Les utilisateurs peuvent créer un compte, s'authentifier de manière sécurisée, consulter les parkings disponibles, démarrer une session de stationnement, effectuer un paiement électronique, recevoir des notifications Push et bénéficier du système de recommandation intelligent. Les propriétaires peuvent proposer leurs parkings privés, tandis que les administrateurs disposent des outils nécessaires pour les valider et assurer la gestion de la plateforme.

Les tests ont également permis de vérifier le bon fonctionnement des échanges entre les différents composants de la solution. Le Backend assure la sécurisation des requêtes grâce à l'authentification JWT, communique correctement avec la base de données PostgreSQL et s'intègre aux différents services externes tels que Flouci, Firebase Cloud Messaging et Cloudinary. Le système de recommandation basé sur le modèle Random Forest fournit des recommandations cohérentes en fonction des caractéristiques des parkings et des préférences de l'utilisateur.

Malgré les résultats obtenus, certaines améliorations peuvent être envisagées. Parmi celles-ci figurent l'optimisation des performances du système de recommandation à partir d'un plus grand volume de données, l'activation de la vérification des numéros de téléphone par SMS via Twilio, ainsi que l'ajout de nouvelles fonctionnalités de gestion et d'analyse destinées aux administrateurs.

5.9 Conclusion

Ce chapitre a présenté les différentes activités de validation réalisées sur l'application **TuniPark**. Il a couvert les tests des API du Backend, les tests fonctionnels, la validation de la base de données ainsi que la vérification des principaux services externes intégrés à la solution.

Les résultats obtenus montrent que les différentes fonctionnalités développées répondent aux exigences définies lors de l'analyse des besoins. Les tests réalisés ont confirmé la fiabilité des principaux modules de l'application, notamment l'authentification, la gestion des parkings, les sessions de stationnement, les paiements électroniques, les notifications Push et le système de recommandation intelligent.

Ce travail fournit un socle robuste pour les futures améliorations de l'application. Les améliorations possibles sont exposées dans la conclusion générale de ce travail de recherche.

Conclusion Générale et Perspectives

Conclusion Générale

Ce projet de fin d'études a permis de concevoir et de développer **TuniPark**, une application intelligente de gestion du stationnement destinée à faciliter la recherche, la réservation et le paiement des places de stationnement. La solution proposée répond aux difficultés rencontrées par les automobilistes lors de la recherche d'une place disponible, tout en offrant aux propriétaires de parkings privés un moyen simple de proposer leurs emplacements et de les gérer efficacement.

Le projet a débuté par une étude du contexte général du stationnement ainsi que par une analyse des besoins des différents acteurs. Cette phase a permis d'identifier les exigences fonctionnelles et non fonctionnelles du système, de définir les cas d'utilisation et d'établir le Product Backlog ayant servi de base à la réalisation de l'application.

La phase de conception a permis de définir l'architecture générale de la solution à travers les architectures logique et physique ainsi que les différents diagrammes UML. Les choix technologiques se sont appuyés sur **Flutter** pour la réalisation de l'application mobile, **NestJS** pour le Backend, **PostgreSQL** pour le stockage des données et **Angular** pour la plateforme web d'administration. Plusieurs services externes ont également été intégrés, notamment **Flouci** pour les paiements électroniques, **Firebase Cloud Messaging** pour les notifications Push et **Cloudinary** pour le stockage des images.

La phase d'implémentation a conduit au développement d'une application complète intégrant l'authentification sécurisée, la gestion des parkings publics et privés, la création des sessions de stationnement, le paiement électronique, les notifications Push ainsi qu'une plateforme web destinée à l'administration des parkings et des utilisateurs. Une attention particulière a également été portée au développement d'un système de recommandation intelligent reposant sur un modèle de **Random Forest**, permettant de proposer aux utilisateurs les parkings les plus adaptés à leur situation.

Enfin, les différentes phases de validation ont confirmé le bon fonctionnement des principaux modules de l'application. Les tests des API, les tests fonctionnels ainsi que la validation des services externes ont permis de vérifier la fiabilité de la solution développée et de s'assurer que les principales exigences définies au début du projet étaient satisfaites.

Perspectives

Plusieurs améliorations pourront être apportées aux futures versions de **Tuni-Park**. Une première évolution consistera à enrichir le système de recommandation en exploitant un volume plus important de données réelles afin d'améliorer la précision des suggestions proposées aux utilisateurs. L'intégration de techniques d'apprentissage plus avancées et la prise en compte des habitudes de stationnement permettraient d'obtenir des recommandations encore plus pertinentes.

L'application pourrait également intégrer de nouveaux moyens de paiement ainsi que l'activation complète de la vérification des numéros de téléphone par SMS à l'aide de **Twilio**. Cette fonctionnalité renforcerait la sécurité des comptes utilisateurs et simplifierait les procédures de récupération d'accès.

Enfin, plusieurs fonctionnalités destinées aux administrateurs et aux propriétaires de parkings pourraient être développées, telles que des tableaux de bord statistiques plus avancés, l'analyse de l'occupation des parkings en temps réel, la génération automatique de rapports ainsi que des outils d'aide à la décision permettant d'optimiser la gestion des espaces de stationnement.

Bibliographie

- [1] K. Beck *et al.*, *Manifesto for Agile Software Development*. Agile Alliance, 2001. [Online]. Available : <https://agilemanifesto.org>
- [2] K. Schwaber and J. Sutherland, *The Scrum Guide*. Scrum.org, 2020. [Online]. Available : <https://scrumguides.org>
- [3] Google LLC. (2026) Flutter – build apps for any screen. [Online]. Available : <https://flutter.dev>
- [4] ——. (2026) Angular – the modern web developer’s platform. [Online]. Available : <https://angular.dev>
- [5] NestJS. (2026) Nestjs – a progressive node.js framework. [Online]. Available : <https://nestjs.com>
- [6] Prisma Data, Inc. (2026) Prisma – next-generation node.js and typescript orm. [Online]. Available : <https://www.prisma.io>
- [7] PostgreSQL Global Development Group. (2026) Postgresql : The world’s most advanced open source relational database. [Online]. Available : <https://www.postgresql.org>
- [8] Cloudinary. (2026) Cloudinary – image and video upload, storage, optimization and cdn. [Online]. Available : <https://cloudinary.com>
- [9] Google LLC. (2026) Firebase cloud messaging documentation. [Online]. Available : <https://firebase.google.com/docs/cloud-messaging>
- [10] Flouci. (2026) Flouci – solution de paiement électronique en tunisie. [Online]. Available : <https://flouci.com>
- [11] Nodemailer. (2026) Nodemailer – send emails with node.js. [Online]. Available : <https://nodemailer.com>
- [12] Twilio Inc. (2026) Twilio – customer engagement platform. [Online]. Available : <https://www.twilio.com>
- [13] S. Ramírez. (2026) Fastapi – modern, fast web framework for building apis with python. [Online]. Available : <https://fastapi.tiangolo.com>
- [14] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

-
- [15] Vercel Inc. (2026) Vercel – frontend cloud platform. [Online]. Available : <https://vercel.com>
 - [16] Render. (2026) Render – cloud application hosting platform. [Online]. Available : <https://render.com>
 - [17] M. Jones, J. Bradley, and N. Sakimura. (2015) RFC 7519 : Json web token (jwt). [Online]. Available : <https://www.rfc-editor.org/rfc/rfc7519>
 - [18] SmartBear Software. (2026) Swagger – api documentation & design tools. [Online]. Available : <https://swagger.io>



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454